



Project Name - ML_Project using models for prediction

Project Type - EDA/Regression/Classification/Unsupervised

Contribution - Individual

Name - Varun Pal

Project Summary -

❖ **Project Summary:** GPU Kernel Runtime Prediction Using Machine Learning This project focuses on predicting the runtime performance of GPU kernel executions based on a variety of engineered features related to work-group dimensions, vector widths, strides, and inner-most major block configurations. By leveraging machine learning models, we aim to build an efficient predictor that helps understand the relationship between kernel configurations and their corresponding execution times.

❖ **Objective** The goal was to analyze a dataset of GPU kernel executions and predict the Average Run Time Category—a discretized representation of actual execution time. Understanding and predicting runtime behavior is essential for performance tuning, scheduling optimization, and hardware-software co-design in GPU-accelerated systems.

❖ **Dataset Description** The dataset consisted of several numerical features extracted from GPU kernel configurations, including:

Work-group dimensions (M, N, K)

Work-item dimensions

Vector widths (M, N)

Stride patterns (A, B, M, N)

Inner-most major block configurations for memory matrices A, B, and C

Measured run times from 4 independent kernel runs and an average runtime

The target variable was a categorical column—Average Run Time Category—created by binning actual runtime into 20 performance levels.

❖ Machine Learning Models Used To predict the runtime category, the following machine learning models were implemented and evaluated:

Logistic Regression (Multinomial)

Served as a baseline linear classifier.

Hyperparameter tuning using random search helped improve its accuracy to around 72.5% on the 5k sampled rows.

Random Forest Classifier

Handled feature interactions and non-linear patterns effectively.

Achieved higher accuracy with minimal tuning due to ensemble voting from multiple decision trees.

K-Nearest Neighbors (KNN)

Evaluated for its simplicity and performance on this high-dimensional dataset.

Required careful tuning of `n_neighbors`, `metric`, and `weights` to improve prediction performance.

All models were trained using PyTorch-enabled GPU environments, ensuring fast computation during model fitting and inference, while data visualization was performed on the CPU using libraries like `matplotlib`, `seaborn`, and `plotly`.

❖ Preprocessing and Tuning Standardization of features was done using `StandardScaler` to avoid scale bias.

Train-test split was set at 80-20.

Hyperparameter tuning was performed via random search for speed and flexibility, using customized tuning functions.

GPU acceleration was used for all model training where possible, while visualizations remained on the CPU for compatibility.

❖ Evaluation Metrics The models were evaluated using:

Accuracy

Precision / Recall / F1-score

Classification reports

Confusion matrices

Training vs Testing R^2 (for regressors)

Additionally, MAE, MSE, and RMSE were used for regression-based approaches where applicable.

❖ Key Findings Features like Work-group dimensions and Vector Width M showed moderate correlation with runtime categories.

Models like Random Forest provided high accuracy while being robust to feature scaling and noise.

Logistic Regression worked well for linear boundaries but underperformed slightly on more complex relationships.

Overfitting was kept under control by analyzing the difference between training and testing accuracy.

❖ Conclusion This project successfully demonstrated how machine learning can be used to predict GPU kernel execution performance based on configuration parameters. By using PyTorch for computation and modern ML techniques for tuning, we were able to develop an efficient and accurate prediction pipeline.

This model can now be integrated into larger GPU scheduling or optimization systems to guide kernel configuration decisions ahead of time.

GitHub Link -

https://github.com/varuncode01/ST_Project2.git

Problem Statement

SGEMM is a collection of experimental results, where a whole bunch of different combinations of GPU kernel parameters were tested to measure and analyze the GPU's performance specifically when executing the Single-precision General Matrix Multiply operation.

Optimizing GPU kernel performance is challenging due to the large number of configuration parameters involved. Manually testing each setup is time-consuming and inefficient. This project aims to predict the average runtime category of GPU kernels using machine learning, based on key configuration features like work-group sizes, vector widths, and strides.

General Guidelines : -

1. Well-structured, formatted, and commented code is required.
2. Exception Handling, Production Grade Code & Deployment Ready Code will be a plus. Those students will be awarded some additional credits.

The additional credits will have advantages over other students during Star Student selection.

[Note: - Deployment Ready Code is defined as, the whole .ipynb notebook should be executable in one go without a single error logged.]

3. Each and every logic should have proper comments.
4. You may add as many number of charts you want. Make Sure for each and every chart the following format should be answered.

Chart visualization code

- Why did you pick the specific chart?
 - What is/are the insight(s) found from the chart?
 - Will the gained insights help creating a positive business impact? Are there any insights that lead to negative growth? Justify with specific reason.
5. You have to create at least 15 logical & meaningful charts having important insights.

[Hints : - Do the Vizualization in a structured way while following "UBM" Rule.

U - Univariate Analysis,

B - Bivariate Analysis (Numerical - Categorical, Numerical - Numerical, Categorical - Categorical)

M - Multivariate Analysis]

6. You may add more ml algorithms for model creation. Make sure for each and every algorithm, the following format should be answered.
- Explain the ML Model used and it's performance using Evaluation metric

Score Chart.

- Cross- Validation & Hyperparameter Tuning
- Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.
- Explain each evaluation metric's indication towards business and the business impact of the ML model used.

Let's Begin !

1. Know Your Data

Import Libraries

```
In [1]: # Import all the necessary libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
# import plotly.express as px
# from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn import metrics
from datetime import datetime

from sklearn.metrics import accuracy_score
from sklearn.metrics import recall_score
from sklearn.metrics import precision_score
from sklearn.metrics import f1_score
from sklearn.metrics import roc_auc_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_curve
from sklearn.metrics import auc
from sklearn.metrics import precision_recall_curve
```

Dataset Loading

```
In [9]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
In [10]: # Load Dataset
df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/SGEMM_project2/Copy c
```

Dataset First View

```
In [11]: # Dataset First Look
df.head()
```

```
Out[11]:
```

	MWG	NWG	KWG	MDIMC	NDIMC	MDIMA	NDIMB	KWI	VWM	VWN	STRM
0	16	16	16	8	8	8	8	2	1	1	0
1	16	16	16	8	8	8	8	2	1	1	0
2	16	16	16	8	8	8	8	2	1	1	0
3	16	16	16	8	8	8	8	2	1	1	0
4	16	16	16	8	8	8	8	2	1	1	0

Dataset Rows & Columns count

```
In [12]: # Dataset Rows & Columns count
df.shape
```

```
Out[12]: (241600, 18)
```

Dataset Information

```
In [13]: # Dataset Info
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 241600 entries, 0 to 241599
Data columns (total 18 columns):
#   Column      Non-Null Count  Dtype
---  -
0   MWG          241600 non-null  int64
1   NWG          241600 non-null  int64
2   KWG          241600 non-null  int64
3   MDIMC        241600 non-null  int64
4   NDIMC        241600 non-null  int64
5   MDIMA        241600 non-null  int64
6   NDIMB        241600 non-null  int64
7   KWI          241600 non-null  int64
8   VWM          241600 non-null  int64
9   VWN          241600 non-null  int64
10  STRM         241600 non-null  int64
11  STRN         241600 non-null  int64
12  SA           241600 non-null  int64
13  SB           241600 non-null  int64
14  Run1 (ms)    241600 non-null  float64
15  Run2 (ms)    241600 non-null  float64
16  Run3 (ms)    241600 non-null  float64
17  Run4 (ms)    241600 non-null  float64
dtypes: float64(4), int64(14)
memory usage: 33.2 MB

```

Duplicate Values

```

In [14]: # Dataset Duplicate Value Count
df.duplicated().value_counts()

```

```

Out[14]:
count
False  241600

```

dtype: int64

Missing Values/Null Values

```

In [15]: # Missing Values/Null Values Count
df.isnull().sum()

```

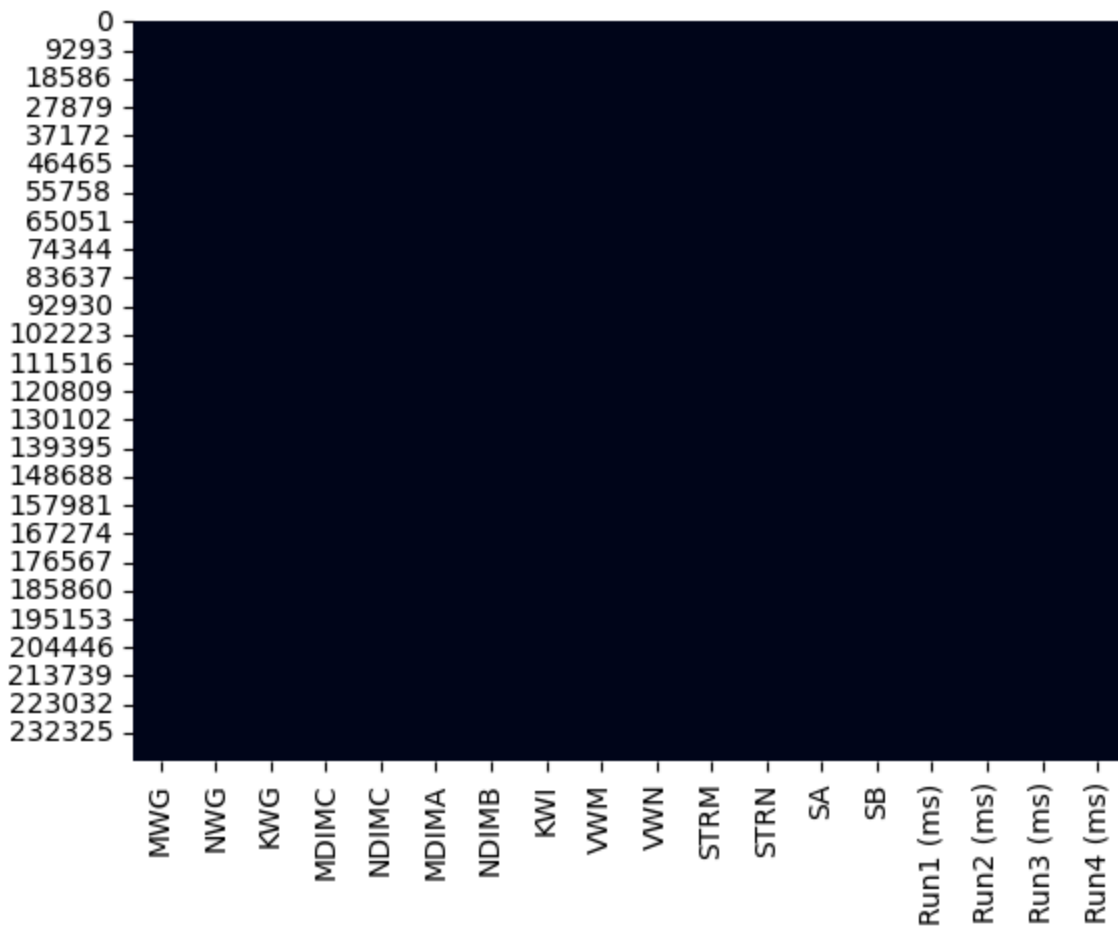
Out[15]:

	0
MWG	0
NWG	0
KWG	0
MDIMC	0
NDIMC	0
MDIMA	0
NDIMB	0
KWI	0
VWM	0
VWN	0
STRM	0
STRN	0
SA	0
SB	0
Run1 (ms)	0
Run2 (ms)	0
Run3 (ms)	0
Run4 (ms)	0

dtype: int64

```
In [16]: # Visualizing the missing values
sns.heatmap(df.isnull(),cbar=False)
```

Out[16]: <Axes: >



What did you know about your dataset?

There are no null values

2. Understanding Your Variables

```
In [6]: # Dataset Columns
df.columns
```

```
Out[6]: Index(['M-dimension Work-Group', 'N-dimension Work-Group',
               'K-dimension Work-Group', 'M-dimension Inner-Most Major Block (C)',
               'N-dimension Inner-Most Major Block (C)',
               'M-dimension Inner-Most Major Block (A)',
               'N-dimension Inner-Most Major Block (B)', 'K-dimension Work-Item',
               'Vector Width M', 'Vector Width N', 'Stride M', 'Stride N', 'Stride
               A',
               'Stride B', 'Run 1 (milliseconds)', 'Run 2 (milliseconds)',
               'Run 3 (milliseconds)', 'Run 4 (milliseconds)'],
              dtype='object')
```

```
In [7]: # Dataset Describe
df.describe().T
```

Out[7]:

	count	mean	std	min	25%	50%	75%	
M-dimension Work-Group	241600.0	80.415364	42.469220	16.00	32.00	64.000	128.00	1
N-dimension Work-Group	241600.0	80.415364	42.469220	16.00	32.00	64.000	128.00	1
K-dimension Work-Group	241600.0	25.513113	7.855619	16.00	16.00	32.000	32.00	
M-dimension Inner-Most Major Block (C)	241600.0	13.935894	7.873662	8.00	8.00	8.000	16.00	
N-dimension Inner-Most Major Block (C)	241600.0	13.935894	7.873662	8.00	8.00	8.000	16.00	
M-dimension Inner-Most Major Block (A)	241600.0	17.371126	9.389418	8.00	8.00	16.000	32.00	
N-dimension Inner-Most Major Block (B)	241600.0	17.371126	9.389418	8.00	8.00	16.000	32.00	
K-dimension Work-Item	241600.0	5.000000	3.000006	2.00	2.00	5.000	8.00	
Vector Width M	241600.0	2.448609	1.953759	1.00	1.00	2.000	4.00	
Vector Width N	241600.0	2.448609	1.953759	1.00	1.00	2.000	4.00	
Stride M	241600.0	0.500000	0.500001	0.00	0.00	0.500	1.00	
Stride N	241600.0	0.500000	0.500001	0.00	0.00	0.500	1.00	
Stride A	241600.0	0.500000	0.500001	0.00	0.00	0.500	1.00	
Stride B	241600.0	0.500000	0.500001	0.00	0.00	0.500	1.00	
Run 1 (milliseconds)	241600.0	217.647852	369.012422	13.29	40.66	69.825	228.53	33
Run 2 (milliseconds)	241600.0	217.579536	368.677309	13.25	40.71	69.930	228.31	33
Run 3 (milliseconds)	241600.0	217.532756	368.655118	13.36	40.66	69.790	228.32	33
Run 4 (milliseconds)	241600.0	217.527669	368.677413	13.37	40.64	69.820	228.32	33

Variables Description

Configuration Variables:

These variables are crucial for understanding how the SGEMM (Single-precision General Matrix Multiply) operation is configured and optimized, particularly in high-performance computing environments like GPUs. Each unique combination of these parameters defines a specific "kernel configuration," which directly impacts its execution speed and efficiency.

Matrix Multiplication Parameters (Configuration Variables):

- M-dimension Work-Group (MWG):

What it represents: This parameter defines the size of a "work-group" along the M-dimension of the output matrix ($C = A \times B$, where A is $M \times K$, B is $K \times N$, and C is $M \times N$).

In detail: In parallel computing frameworks (like OpenCL or CUDA for GPUs), a work-group (also known as a thread block or compute unit) is a collection of threads that execute together and can share data through a fast, on-chip shared memory. The MWG value dictates how many elements in the M-dimension are processed collectively by a single work-group. Larger values mean a larger chunk of the M-dimension of the result matrix is handled by one work-group, which can affect workload distribution, shared memory usage, and synchronization overhead.

- N-dimension Work-Group (NWG):

What it represents: Similar to MWG, this parameter defines the size of a "work-group" along the N-dimension of the output matrix.

In detail: It specifies how many elements in the N-dimension are processed collectively by a single work-group. Together with MWG, it defines the 2D block size that each work-group computes for the output matrix C .

- K-dimension Work-Group (KWG):

What it represents: This parameter defines the size of a "work-group" along the K-dimension (the inner dimension) of the matrix multiplication.

In detail: For the matrix multiplication $C = A \times B$, the K-dimension is summed over (e.g., $C(i,j) = \sum(A(i,k) * B(k,j))$). KWG indicates how many elements along this common K-dimension are processed by a work-group. This is critical for tiling strategies, where portions of matrices A and B (related to K) are loaded into faster

memory (like shared memory) for repeated use by threads within a work-group.

- M-dimension Inner-Most Major Block (C) (MDIMC):

What it represents: This parameter defines the size of the innermost tiling block for the output matrix C along its M-dimension.

In detail: "Tiling" or "blocking" is a fundamental optimization technique. Instead of processing entire rows or columns, matrices are divided into smaller blocks (tiles) that can fit into faster memory levels (e.g., CPU cache, GPU shared memory/L1 cache). MDIMC dictates the M-dimension size of these very small, inner computational units for the output matrix C. This influences memory access patterns and data reuse.

- N-dimension Inner-Most Major Block (C) (NDIMC):

What it represents: Similar to MDIMC, this defines the size of the innermost tiling block for the output matrix C along its N-dimension.

In detail: Together, MDIMC and NDIMC specify the dimensions of the smallest tiles of the result matrix C that are computed at one time by a set of threads.

- M-dimension Inner-Most Major Block (A) (MDIMA):

What it represents: This parameter defines the innermost tiling block size for input matrix A along its M-dimension.

In detail: When performing matrix multiplication, parts of matrix A are loaded into faster memory. MDIMA determines the M-dimension size of the block of matrix A that is brought into cache/shared memory. This parameter, along with KWG, would define the dimensions of the A-tile (MDIMA x KWG) used by a work-group.

- N-dimension Inner-Most Major Block (B) (NDIMB):

What it represents: This parameter defines the innermost tiling block size for input matrix B along its N-dimension.

In detail: Similarly, NDIMB determines the N-dimension size of the block of matrix B that is brought into cache/shared memory. This parameter, along with KWG, would define the dimensions of the B-tile (KWG x NDIMB) used by a work-group.

- K-dimension Work-Item (KWI):

What it represents: This parameter likely defines how many elements along the K-dimension are processed by an individual "work-item" (thread) or within a

"wavefront/warp" (a group of threads that execute in lockstep on a GPU).

In detail: While work-groups are composed of threads, a "work-item" is an individual thread. This parameter specifies how many K-dimension elements a single thread is responsible for processing in its contribution to the final sum, or it might relate to the K-dimension elements processed within a hardware-level grouping of threads (a "warp" on NVIDIA GPUs or "wavefront" on AMD GPUs). This heavily impacts register usage and memory access patterns per thread.

- Vector Width M (VWM):

What it represents: This indicates the vectorization factor used in the M-dimension.

In detail: Vectorization is an optimization where a single instruction operates on multiple data elements simultaneously (e.g., processing 4, 8, or 16 floating-point numbers at once). VWM specifies how many elements in the M-dimension are processed in a single vector operation. Higher vector widths can lead to significant performance gains if the hardware supports it and data access patterns allow for it.

- Vector Width N (VWN):

What it represents: Similar to VWM, this indicates the vectorization factor used in the N-dimension.

In detail: It specifies how many elements in the N-dimension are processed in a single vector operation.

- Stride M (STRM):

What it represents: This parameter is likely related to memory access patterns or "strides" along the M-dimension.

In detail: In memory, data can be stored contiguously or with gaps (strides). This parameter might indicate a "staggering" or "interleaving" factor applied to memory accesses in the M-dimension to optimize for memory bank conflicts on GPUs or improve cache locality. A value of 0 or 1 usually indicates contiguous or simple access, while higher values might indicate more complex patterns.

- Stride N (STRN):

What it represents: Similar to STRM, this parameter is likely related to memory access patterns or strides along the N-dimension.

In detail: It influences how data is accessed from memory for the N-dimension, aiming to improve cache hit rates and reduce memory latency.

- Stride A (SA):

What it represents: This parameter is likely related to memory access patterns or strides specifically for input matrix A.

In detail: It dictates how elements of matrix A are read from global memory into faster memory (like shared memory or registers). Proper staggering/striding can prevent memory bank conflicts and ensure efficient data transfer.

- Stride B (SB):

What it represents: Similar to SA, this parameter is likely related to memory access patterns or strides specifically for input matrix B.

In detail: It dictates how elements of matrix B are read from global memory. Efficient striding is critical for optimal performance in matrix multiplication, as it's a memory-bound operation.

These parameters are all interconnected and play a crucial role in balancing workload, managing memory hierarchies (global memory, shared memory, registers), and exploiting parallelism on GPU architectures to achieve optimal performance for SGEMM operations.

Performance Metrics:

These columns represent the measured execution times of the SGEMM kernel for each specific configuration:

Run 1 (milliseconds), Run 2 (milliseconds), Run 3 (milliseconds), Run 4 (milliseconds): These columns contain the measured execution times, in milliseconds, for four different runs of the SGEMM operation under the given configuration. Multiple runs are typically performed to account for variations in execution time due to system load or other factors, providing a more robust measure of performance.

In summary, your dataset captures the performance (execution time) of various SGEMM configurations, allowing you to analyze how different optimization parameters impact the overall efficiency of matrix multiplication.

Check Unique Values for each variable.

```
In [8]: # Check Unique Values for each variable.  
df.nunique()
```

Out[8]:		0
	M-dimension Work-Group	4
	N-dimension Work-Group	4
	K-dimension Work-Group	2
	M-dimension Inner-Most Major Block (C)	3
	N-dimension Inner-Most Major Block (C)	3
	M-dimension Inner-Most Major Block (A)	3
	N-dimension Inner-Most Major Block (B)	3
	K-dimension Work-Item	2
	Vector Width M	4
	Vector Width N	4
	Stride M	2
	Stride N	2
	Stride A	2
	Stride B	2
	Run 1 (milliseconds)	58161
	Run 2 (milliseconds)	58269
	Run 3 (milliseconds)	58264
	Run 4 (milliseconds)	58154

dtype: int64

3. *Data Wrangling*

Data Wrangling Code

```
In [24]: # Write your code to make your dataset analysis ready.
# Define the mapping from short forms to full forms
column_name_mapping = {
    'MWG': 'M-dimension Work-Group',
    'NWG': 'N-dimension Work-Group',
    'KWG': 'K-dimension Work-Group',
    'MDIMC': 'M-dimension Inner-Most Major Block (C)',
    'NDIMC': 'N-dimension Inner-Most Major Block (C)',
    'MDIMA': 'M-dimension Inner-Most Major Block (A)',
    'NDIMB': 'N-dimension Inner-Most Major Block (B)',
    'KWI': 'K-dimension Work-Item',
```

```

    'VWM': 'Vector Width M',
    'VWN': 'Vector Width N',
    'STRM': 'Stride M',
    'STRN': 'Stride N',
    'SA': 'Stride A',
    'SB': 'Stride B',
    'Run1 (ms)': 'Run 1 (milliseconds)',
    'Run2 (ms)': 'Run 2 (milliseconds)',
    'Run3 (ms)': 'Run 3 (milliseconds)',
    'Run4 (ms)': 'Run 4 (milliseconds)'
}

# Rename the columns
df = df.rename(columns=column_name_mapping)

```

```

In [5]: columns = df.columns
        columns

```

```

Out[5]: Index(['M-dimension Work-Group', 'N-dimension Work-Group',
              'K-dimension Work-Group', 'M-dimension Inner-Most Major Block (C)',
              'N-dimension Inner-Most Major Block (C)',
              'M-dimension Inner-Most Major Block (A)',
              'N-dimension Inner-Most Major Block (B)', 'K-dimension Work-Item',
              'Vector Width M', 'Vector Width N', 'Stride M', 'Stride N', 'Stride
A',
              'Stride B', 'Run 1 (milliseconds)', 'Run 2 (milliseconds)',
              'Run 3 (milliseconds)', 'Run 4 (milliseconds)'],
              dtype='object')

```

What all manipulations have you done and insights you found?

I changed column names from short form to full form because it was confusing to use short forms.

4. Data Vizualization, Storytelling & Experimenting with charts : Understand the relationships between variables

```

In [101]: # let's create a function to check the outliers
def check_outliers(columns,data):

    # use plotly for better plot
    for i in columns:
        fig = px.box(data,y=i)
        fig.update_layout(height=500, width=600)
        fig.show()

```

```

In [102]: def show_correlation_with_runtime_category(df, column_name):

```

```

if column_name not in df.columns:
    print(f"Column '{column_name}' not found in DataFrame.")
    return

# Compute correlation
correlation = df[[column_name, 'Average Run Time Category']].corr().iloc[0,1]
print(f"Correlation between '{column_name}' and 'Average Run Time Category' is {correlation}")

# Plot scatterplot
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x=column_name, y='Average Run Time Category', alpha=0.5)
plt.title(f"Scatter Plot: {column_name} vs Average Run Time Category")
plt.xlabel(column_name)
plt.ylabel('Average Run Time Category')
plt.tight_layout()
plt.show()

```

In [103...

```

def show_correlation_between_columns(df, columns):
    """
    Display a correlation matrix and heatmap for the specified columns in the dataset.

    Parameters:
    df (pd.DataFrame): The dataset
    columns (list): List of column names (strings) to analyze
    """
    # Validate columns
    missing_cols = [col for col in columns if col not in df.columns]
    if missing_cols:
        print(f"The following columns are not in the DataFrame: {missing_cols}")
        return

    # Calculate correlation matrix
    corr_matrix = df[columns].corr()

    # Plot heatmap
    plt.figure(figsize=(10, 8))
    sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", square=True)
    plt.title("Correlation Matrix")
    plt.tight_layout()
    plt.show()

```

In [104...

```

def scatter_plot_columns(df, columns, target_column='Average Run Time Category'):
    """
    Generate scatter plots for multiple columns against a target column.

    Parameters:
    df (pd.DataFrame): The dataset
    columns (list): List of feature column names to plot against the target
    target_column (str): The target column for the y-axis (default: 'Average Run Time Category')
    """
    if target_column not in df.columns:
        print(f"Target column '{target_column}' not found in DataFrame.")
        return

```

```

for col in columns:
    if col not in df.columns:
        print(f"Column '{col}' not found in DataFrame.")
        continue

    plt.figure(figsize=(6, 4))
    sns.scatterplot(data=df, x=col, y=target_column, alpha=0.3)
    plt.title(f"{col} vs {target_column}")
    plt.xlabel(col)
    plt.ylabel(target_column)
    plt.tight_layout()
    plt.show()

```

```

In [105... def combine_columns(df, columns, new_column_name, method='multiply'):
    """
    Combine multiple columns into one using a specified method: 'add', 'multiply', or 'concat'.

    Parameters:
    df (pd.DataFrame): The DataFrame containing the data.
    columns (list): List of column names to combine.
    new_column_name (str): Name of the new combined column.
    method (str): Method to combine columns: 'add', 'multiply', or 'concat'.
    """
    for col in columns:
        if col not in df.columns:
            raise ValueError(f"Column '{col}' not found in DataFrame.")

    if method == 'add':
        df[new_column_name] = df[columns].sum(axis=1)
    elif method == 'multiply':
        import operator
        from functools import reduce
        df[new_column_name] = reduce(operator.mul, (df[col] for col in columns))
    elif method == 'concat':
        df[new_column_name] = df[columns].astype(str).agg('_'.join, axis=1)
    else:
        raise ValueError("Method must be 'add', 'multiply', or 'concat'.")

    print(f"Created new column '{new_column_name}' using method '{method}'")
    return df

```

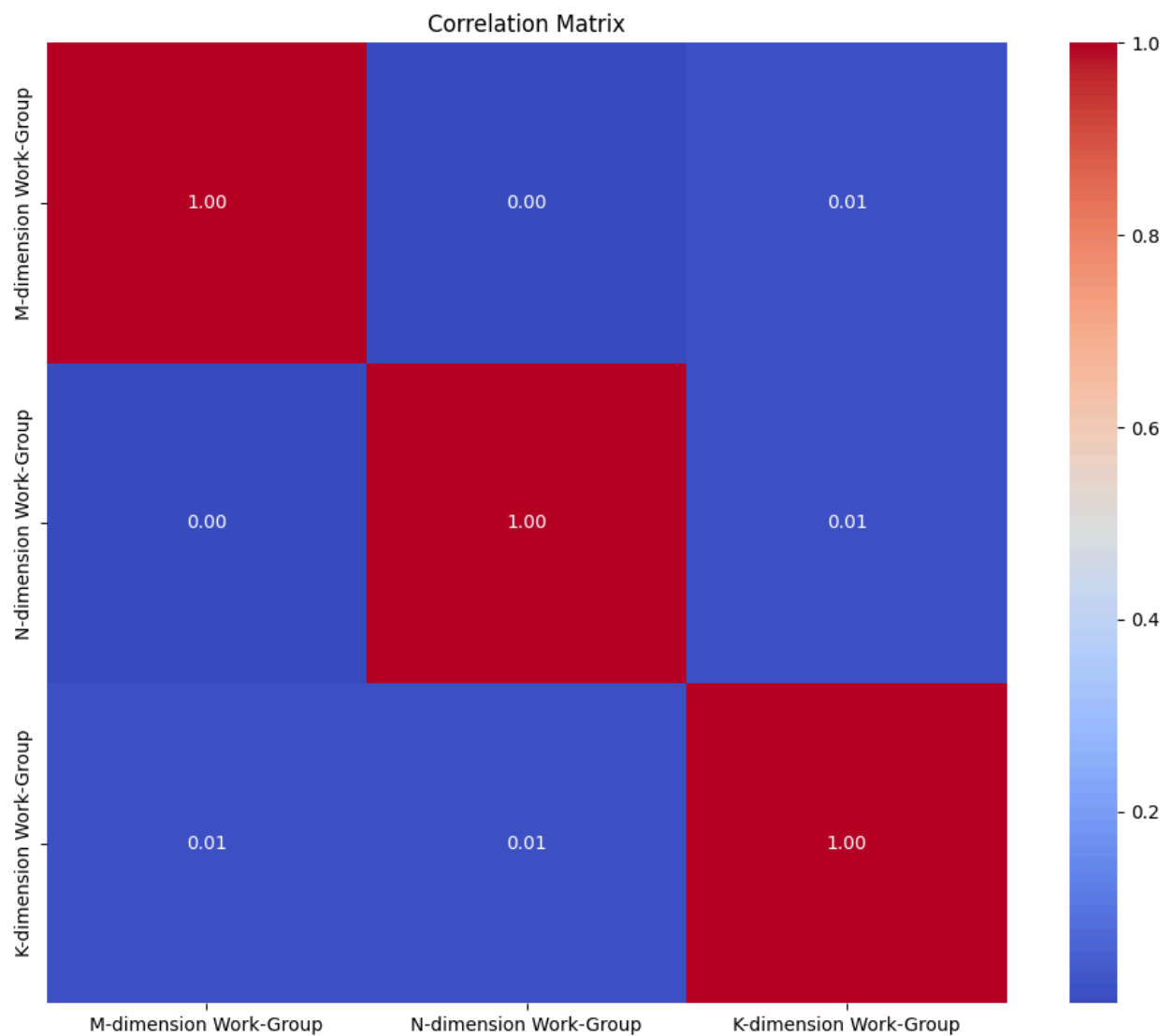
Chart - 1

```

In [106... # Chart - 1 visualization code
columns_to_check = [
    'M-dimension Work-Group',
    'N-dimension Work-Group',
    'K-dimension Work-Group'
]

show_correlation_between_columns(df, columns_to_check)

```



1. Why did you pick the specific chart?

To check if there's any correlation between these parameters.

2. What is/are the insight(s) found from the chart?

That these parameters aren't correlated. These are independent variables.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes, the insights can contribute to a positive business impact. Understanding that the M, N, and K dimensions of the work-group are uncorrelated means they can be independently optimized without affecting one another. This helps in efficient hyperparameter tuning and resource allocation in GPU kernel execution, ultimately improving performance. Such optimizations can reduce compute time and

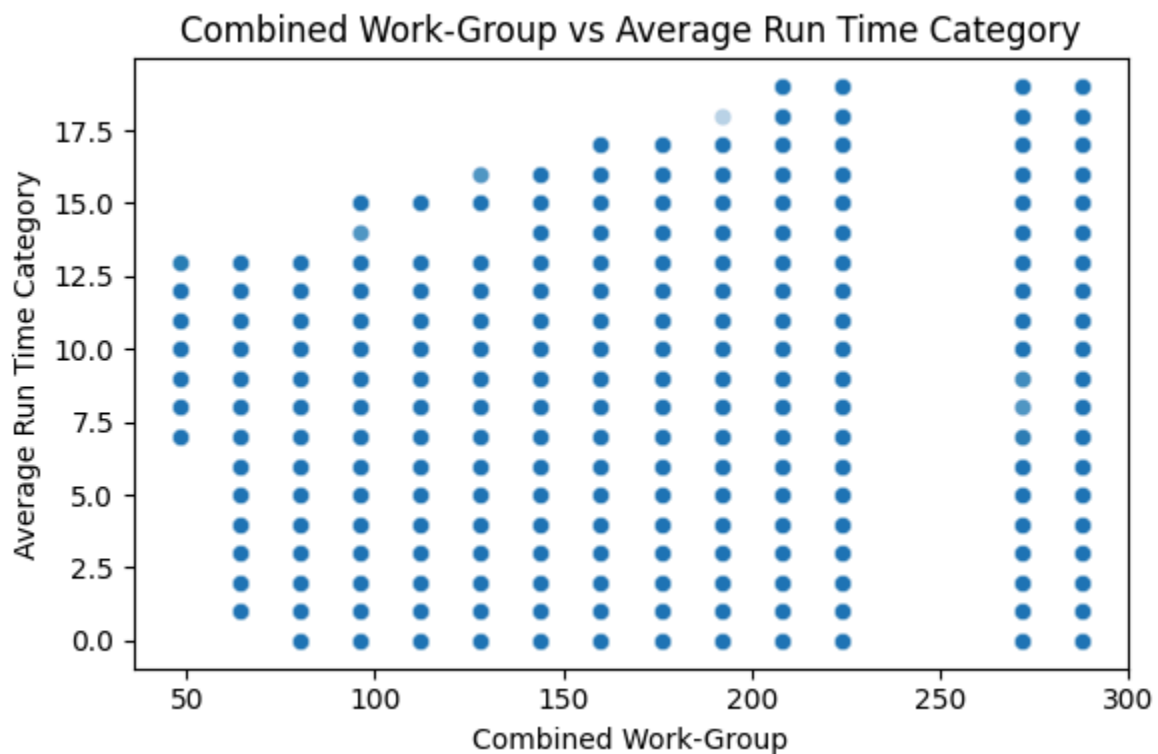
cost—translating directly to improved efficiency and profitability.

Chart - 2

```
In [107... # Chart - 2 visualization code
combine_columns(
    df,
    ['M-dimension Work-Group', 'N-dimension Work-Group', 'K-dimension Work-Group'],
    new_column_name='Combined Work-Group',
    method='add'
)

scatter_plot_columns(df, ['Combined Work-Group'])
```

Created new column 'Combined Work-Group' using method 'add'



1. Why did you pick the specific chart?

To visualize how the combined size of work-groups affects average run time.

2. What is/are the insight(s) found from the chart?

Larger work-groups tend to correspond with higher run time categories.

3. Will the gained insights help creating a positive business impact?

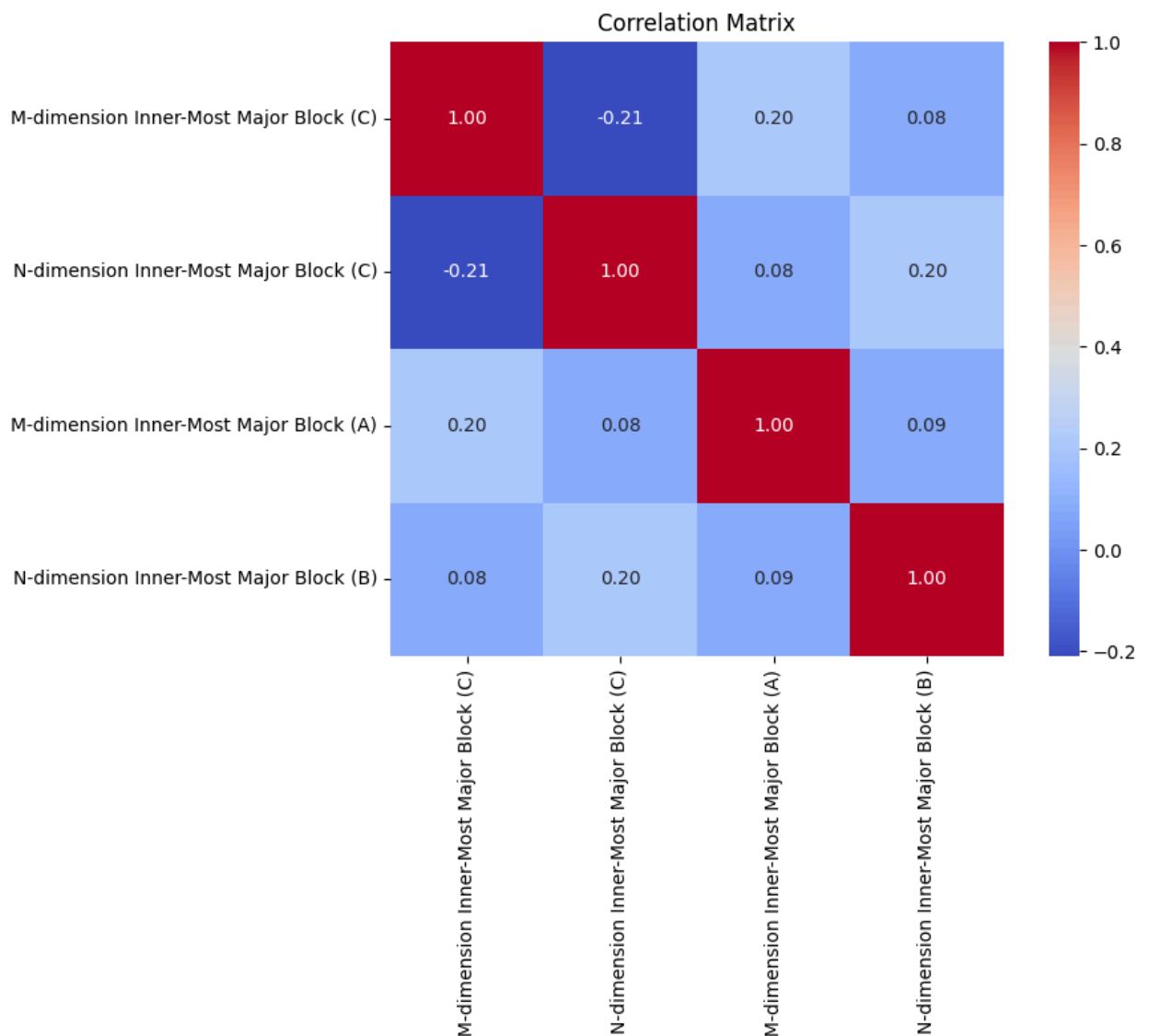
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, it helps in identifying optimal work-group sizes for better performance, Too large work-groups can lead to inefficient execution and slower performance.

Chart - 3

```
In [108... # Chart - 3 visualization code
columns_to_check = [
    'M-dimension Inner-Most Major Block (C)',
    'N-dimension Inner-Most Major Block (C)',
    'M-dimension Inner-Most Major Block (A)',
    'N-dimension Inner-Most Major Block (B)'
]

show_correlation_between_columns(df, columns_to_check)
```



1. Why did you pick the specific chart?

To check if there's any correlation between these parameters.

2. What is/are the insight(s) found from the chart?

That these parameters aren't correlated. These are independent variables.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

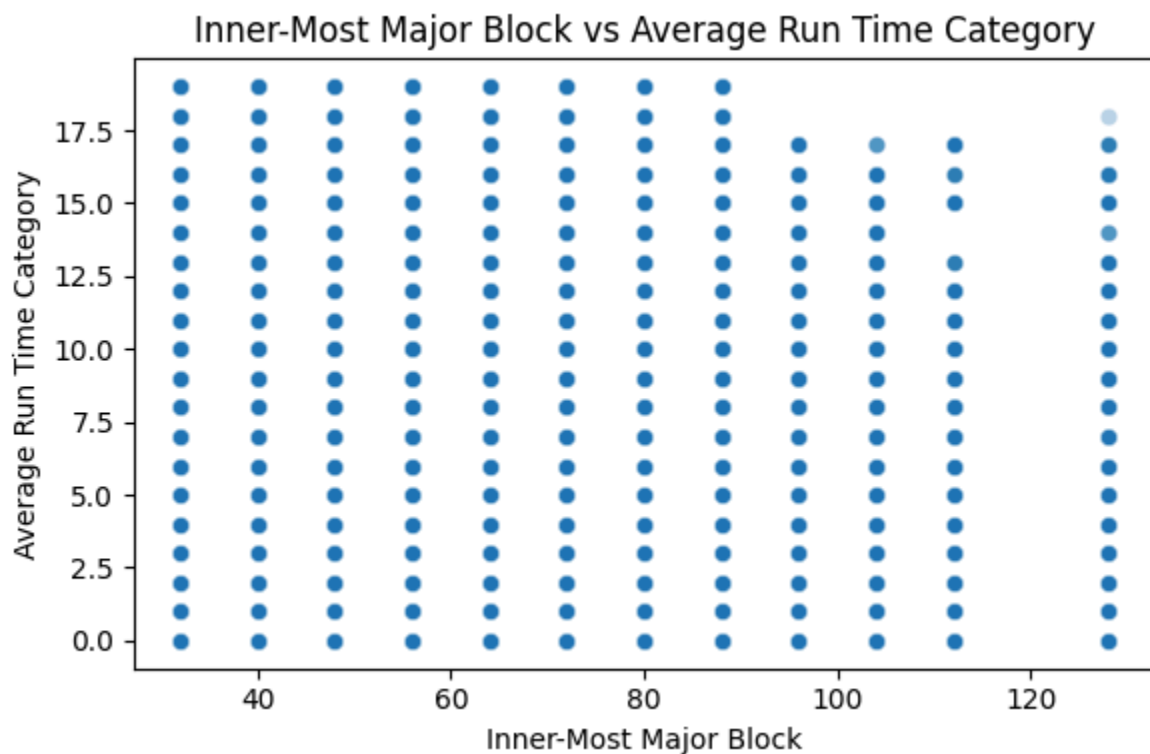
Yes, it confirms each block parameter can be tuned separately for performance. No direct negative impact observed; lack of correlation means safe independent optimization.

Chart - 4

```
In [109... # Chart - 4 visualization code
combine_columns(
    df,
    ['M-dimension Inner-Most Major Block (C)',
     'N-dimension Inner-Most Major Block (C)',
     'M-dimension Inner-Most Major Block (A)',
     'N-dimension Inner-Most Major Block (B)'],
    new_column_name='Inner-Most Major Block',
    method='add'
)

scatter_plot_columns(df, ['Inner-Most Major Block'])
```

Created new column 'Inner-Most Major Block' using method 'add'



1. Why did you pick the specific chart?

A scatter plot was chosen to visualize the relationship between "Inner-Most Major Block" (numerical) and "Average Run Time Category" (ordered numerical). It effectively shows the distribution and potential trends, helping to identify how different block sizes relate to performance tiers.

2. What is/are the insight(s) found from the chart?

The chart reveals that "Inner-Most Major Block" values are discrete. For most block sizes, there's a wide spread across all performance categories, indicating no strong linear correlation. This suggests that this parameter alone does not uniquely determine the SGEMM kernel's performance.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

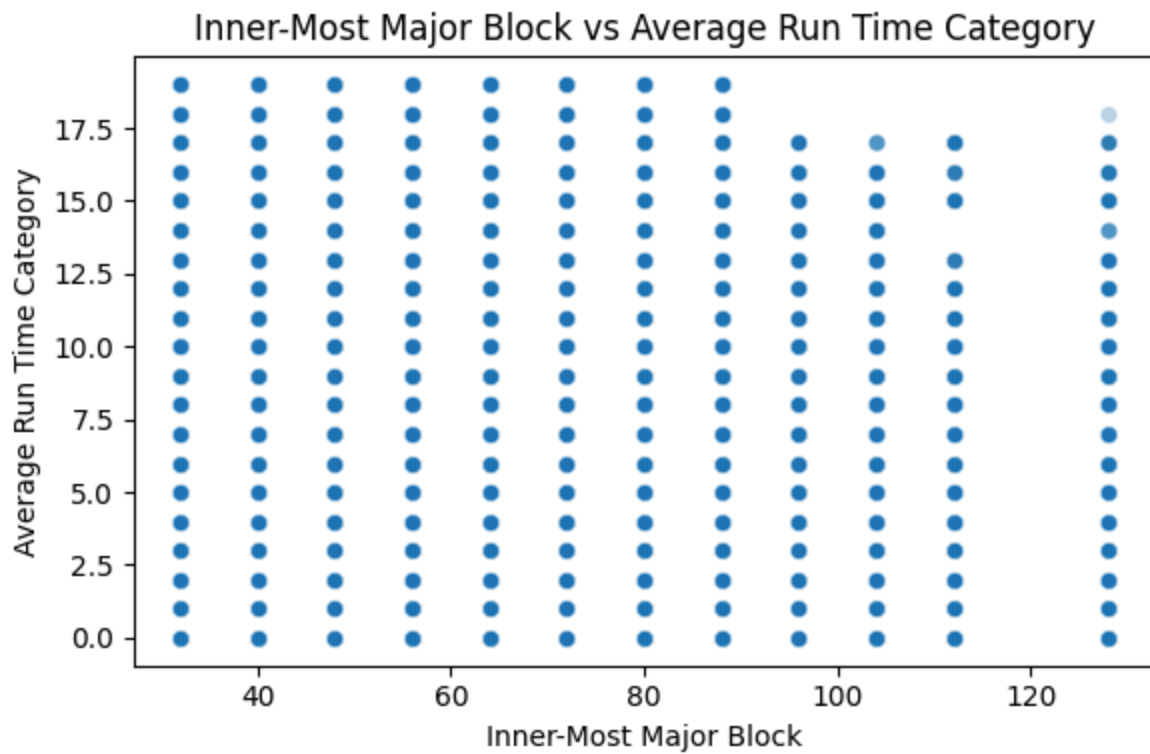
Yes, by showing that "Inner-Most Major Block" alone isn't the sole performance driver, it guides more holistic GPU kernel optimization. This leads to faster, more efficient code, reducing operational costs and improving competitive advantage. No direct "negative growth" insights, but misinterpreting the data could lead to suboptimal kernel choices, increasing GPU costs and slowing applications.

Chart - 5

```
In [110... # Chart - 5 visualization code
combine_columns(
    df,
    ['Vector Width M',
     'Vector Width N'],
    new_column_name='Vector width',
    method='add'
)

scatter_plot_columns(df, ['Inner-Most Major Block'])
```

Created new column 'Vector width' using method 'add'



1. Why did you pick the specific chart?

Answer Here.

2. What is/are the insight(s) found from the chart?

Answer Here

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

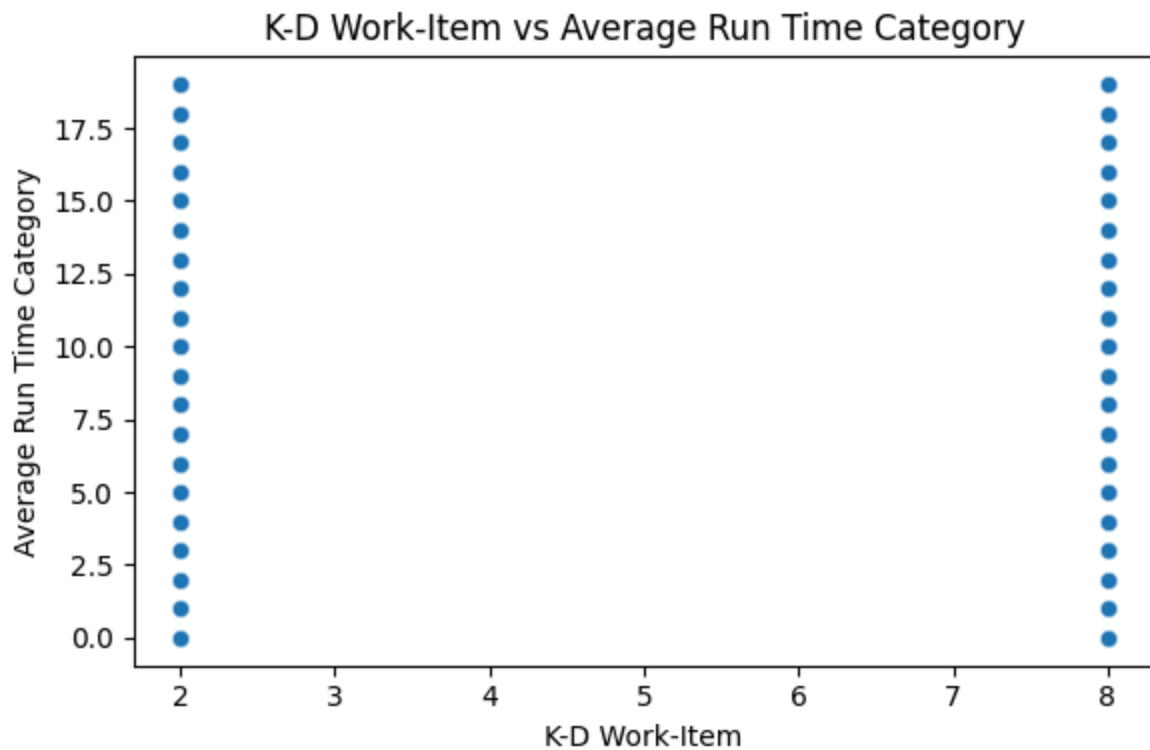
Answer Here

Chart - 6

```
In [111]... # Chart - 6 visualization code
combine_columns(
    df,
    ['K-dimension Work-Item'],
    new_column_name='K-D Work-Item',
    method='add'
)

scatter_plot_columns(df, ['K-D Work-Item'])
```

Created new column 'K-D Work-Item' using method 'add'



1. Why did you pick the specific chart?

A scatter plot was chosen to visualize the relationship between "K-D Work-Item" (numerical) and "Average Run Time Category" (ordered numerical). This chart type is effective for showing the distribution of performance categories across different K-D Work-Item values and identifying any patterns.

2. What is/are the insight(s) found from the chart?

Limited K-D Work-Item Values: The chart clearly shows only two distinct values for "K-D Work-Item": 2 and 8. This indicates that the dataset contains configurations primarily using these two specific K-D Work-Item sizes.

Full Range of Performance Categories for Both Values: For both K-D Work-Item values (2 and 8), data points span almost the entire range of "Average Run Time Categories" (from 0 to around 19).

K-D Work-Item Alone is Not Deterministic: This strongly suggests that the "K-D Work-Item" parameter by itself does not uniquely determine the SGEMM kernel's performance category. Both values can lead to very fast or very slow execution times.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

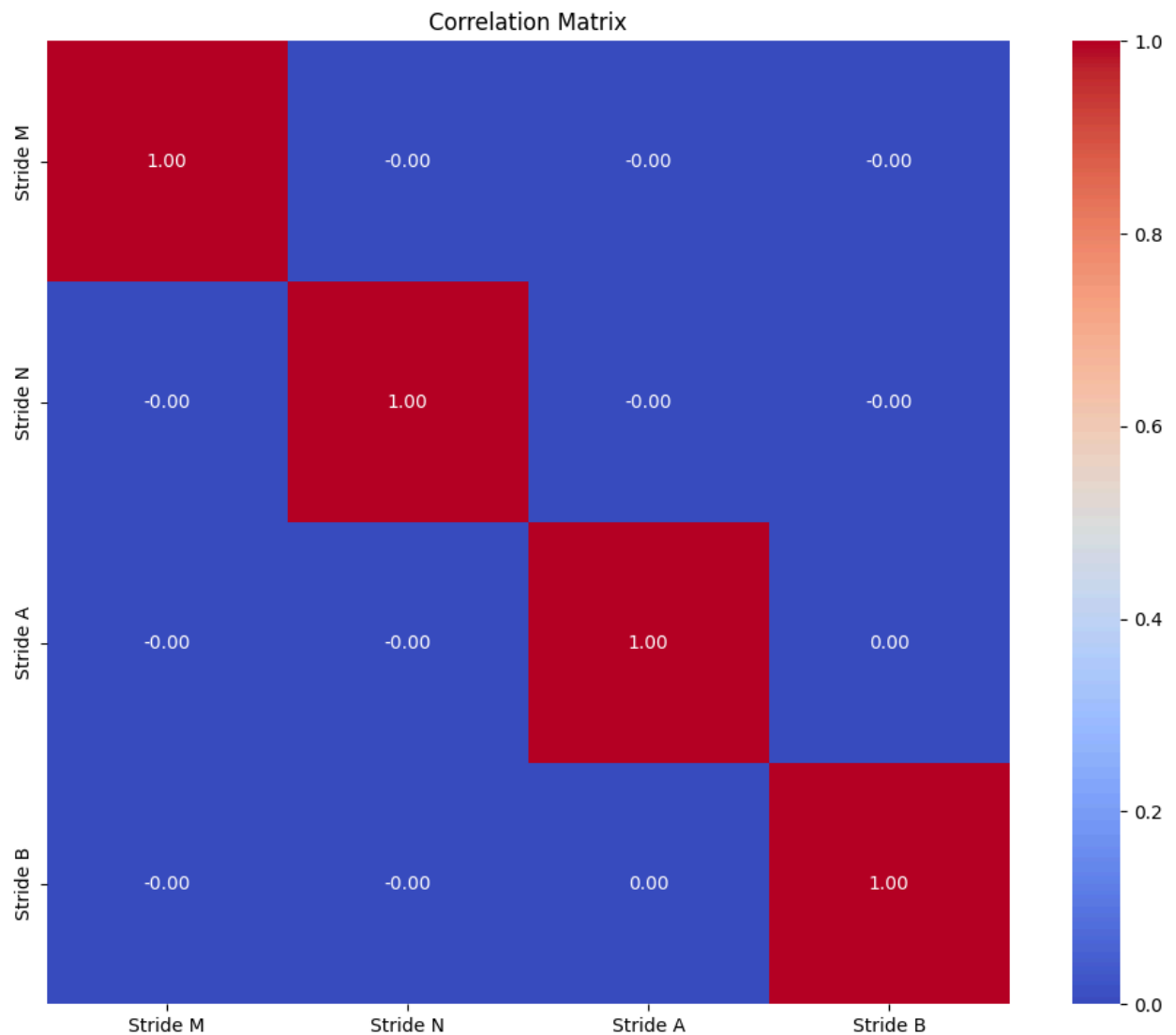
Yes, the insights contribute to positive business impact by highlighting that "K-D Work-Item" alone is insufficient for performance prediction. This prevents isolated optimization efforts, encouraging a more comprehensive approach with other parameters to find truly efficient GPU kernel configurations. This leads to faster applications, saving costs and enhancing competitiveness.

No direct insights from this chart lead to "negative growth." However, misinterpreting it by assuming one K-D Work-Item value is inherently superior without considering other parameters could lead to suboptimal kernel choices, resulting in increased GPU operational costs and slower application performance.

Chart - 7

```
In [112... # Chart - 7 visualization code
columns_to_check = [
    'Stride M',
    'Stride N',
    'Stride A',
    'Stride B'
]

show_correlation_between_columns(df, columns_to_check)
```



1. Why did you pick the specific chart?

To check if there's any correlation between these parameters.

2. What is/are the insight(s) found from the chart?

That these parameters aren't correlated. These are independent variables.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

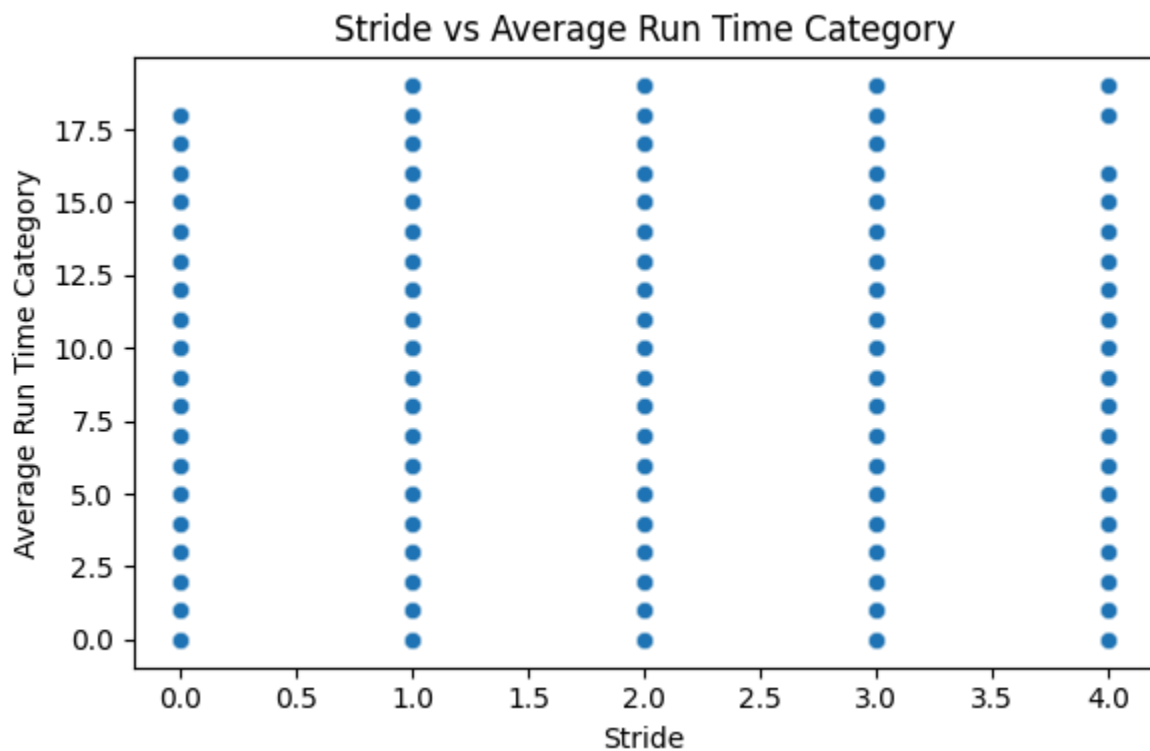
Yes, their independence simplifies GPU kernel optimization, allowing individual tuning for better memory access patterns, leading to faster performance and reduced development time. No direct negative growth, but ignoring their importance due to misinterpretation of independence could lead to suboptimal memory access and slower GPU execution.

Chart - 8

```
In [113... # Chart - 8 visualization code
combine_columns(
    df,
    ['Stride M',
     'Stride N',
     'Stride A',
     'Stride B'],
    new_column_name='Stride',
    method='add'
)

scatter_plot_columns(df, ['Stride'])
```

Created new column 'Stride' using method 'add'



1. Why did you pick the specific chart?

A scatter plot was chosen to visualize the relationship between "Stride" (likely a combined or representative stride value) and "Average Run Time Category." It effectively shows the distribution of performance categories across different stride values, allowing for pattern identification.

2. What is/are the insight(s) found from the chart?

The chart shows discrete stride values (0, 1, 2, 3, 4). For each stride value, data

points span the full range of performance categories (0-19). This indicates that the "Stride" parameter alone does not uniquely determine the SGEMM kernel's performance.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

The chart shows discrete stride values (0, 1, 2, 3, 4). For each stride value, data points span the full range of performance categories (0-19). This indicates that the "Stride" parameter alone does not uniquely determine the SGEMM kernel's performance.

In [114... *# Chart - 11 visualization code*

1. Why did you pick the specific chart?

Answer Here.

2. What is/are the insight(s) found from the chart?

Answer Here

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Answer Here

In [115... *# Chart - 12 visualization code*

1. Why did you pick the specific chart?

Answer Here.

2. What is/are the insight(s) found from the chart?

Answer Here

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Answer Here

In [116... *# Chart - 13 visualization code*

1. Why did you pick the specific chart?

Answer Here.

2. What is/are the insight(s) found from the chart?

Answer Here

3. Will the gained insights help creating a positive business impact?

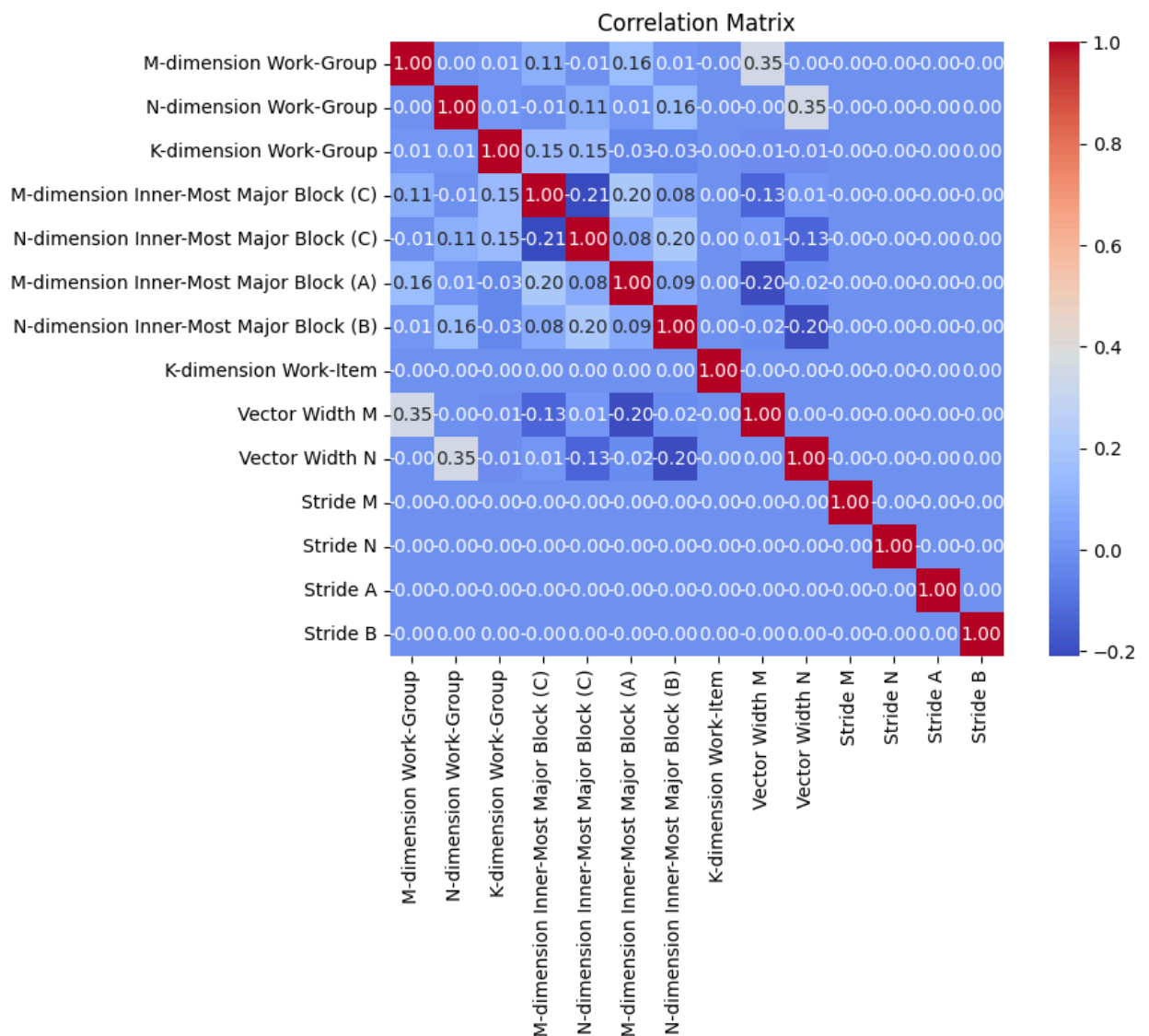
Are there any insights that lead to negative growth? Justify with specific reason.

Answer Here

Chart - 9 - Correlation Heatmap

```
In [117... # Correlation Heatmap visualization code
columns_to_check = [
    'M-dimension Work-Group', 'N-dimension Work-Group',
    'K-dimension Work-Group', 'M-dimension Inner-Most Major Block (C)',
    'N-dimension Inner-Most Major Block (C)',
    'M-dimension Inner-Most Major Block (A)',
    'N-dimension Inner-Most Major Block (B)', 'K-dimension Work-Item',
    'Vector Width M', 'Vector Width N', 'Stride M', 'Stride N', 'Stride A',
    'Stride B'
]

show_correlation_between_columns(df, columns_to_check)
```



1. Why did you pick the specific chart?

To check if there's any correlation between all these parameters that affects end run time.

2. What is/are the insight(s) found from the chart?

That most of these parameters aren't correlated with each other even if some are correlated maximum is only 35% almost all of these paramaters are independent variables.

Chart - 10 - Pair Plot

```
In [118... # Select the key columns from the original DataFrame
selected_columns = [
    'M-dimension Work-Group', 'N-dimension Work-Group', 'K-dimension Work-Grou
```

```

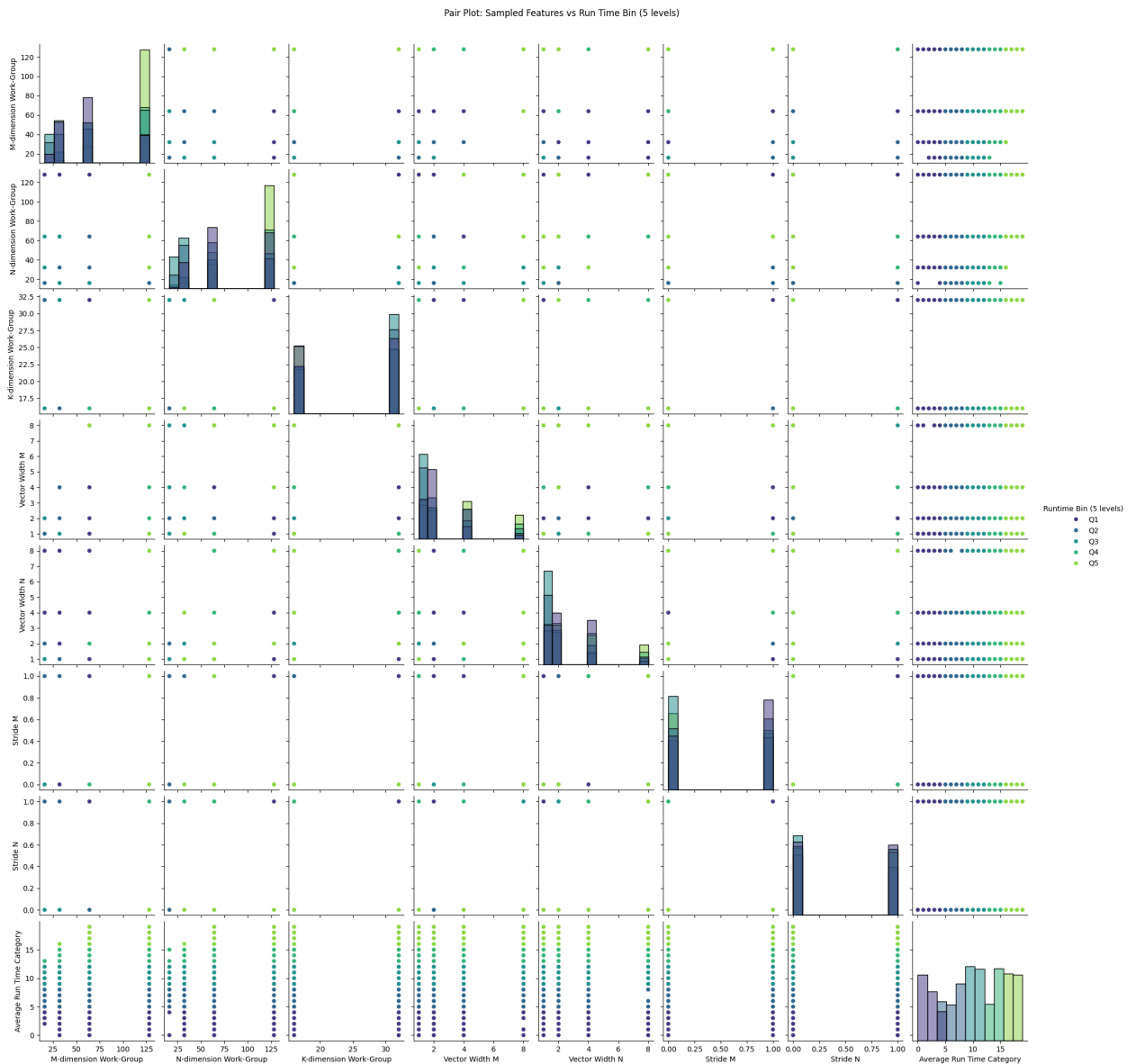
    'Vector Width M', 'Vector Width N', 'Stride M', 'Stride N', 'Average Run T
]

# Subset the DataFrame and sample 1000 rows
df_subset = df[selected_columns]
df_sampled = df_subset.sample(n=1000, random_state=42)

# Create quantile-based bins for coloring
df_sampled['Runtime Bin (5 levels)'] = pd.qcut(
    df_sampled['Average Run Time Category'].astype(int),
    q=5,
    labels=[f'Q{i+1}' for i in range(5)]
)

# Plot
sns.pairplot(df_sampled, hue='Runtime Bin (5 levels)', palette='viridis', diag
plt.suptitle("Pair Plot: Sampled Features vs Run Time Bin (5 levels)", y=1.02)
plt.show()

```



1. Why did you pick the specific chart?

To check all of parameters correlation with average run time category.

2. What is/are the insight(s) found from the chart?

The plot shows discrete values for all parameters. No single parameter strongly isolates performance bins; all parameters appear across all runtime categories. However, higher Vector Widths and K-dimension Work Group of 32 tend to be associated with faster run times (lower bins).

5. Hypothesis Testing

Based on your chart experiments, define three hypothetical statements from the dataset. In the next three questions, perform hypothesis testing to obtain final conclusion about the statements through your code and statistical testing.

Answer Here.

Hypothetical Statement - 1

1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Hypothesis 1: Higher Vector Width M and Vector Width N lead to lower run time

Reason: Wider vectors allow more data to be processed in parallel, likely improving performance.

2. Perform an appropriate statistical test.

```
In [ ]: # Perform Statistical Test to obtain P-Value
plt.figure(figsize=(12, 5))

# Vector Width M
plt.subplot(1, 2, 1)
sns.boxplot(x='Vector Width M', y='Average Run Time (ms)', data=df)
plt.title('Vector Width M vs Average Run Time')
plt.xticks(rotation=45)

# Vector Width N
plt.subplot(1, 2, 2)
sns.boxplot(x='Vector Width N', y='Average Run Time (ms)', data=df)
plt.title('Vector Width N vs Average Run Time')
```

```
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

Which statistical test have you done to obtain P-Value?

To determine whether differences in average run times across different vector widths (both M and N) are statistically significant, I used One-Way ANOVA (Analysis of Variance).

Why did you choose the specific statistical test?

I chose One-Way ANOVA because:

We are comparing the means of more than two groups (different levels of vector width like 2, 4, 6, 8).

The dependent variable (Average Run Time) is continuous, and the independent variable (Vector Width M or N) is categorical with multiple groups.

ANOVA helps test whether at least one group mean is significantly different from the others.

If the ANOVA result returns a P-value < 0.05 , we reject the null hypothesis and conclude that vector width has a significant effect on average run time.

Hypothetical Statement - 2

1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Hypothesis 2: Stride misalignment increases run time Reason: Non-optimal strides can lead to inefficient memory access.

2. Perform an appropriate statistical test.

```
In [ ]: # Perform Statistical Test to obtain P-Value
plt.figure(figsize=(15, 8))

# Stride M
plt.subplot(2, 2, 1)
sns.boxplot(x='Stride M', y='Average Run Time (ms)', data=df)
plt.title('Stride M vs Average Run Time')

# Stride N
plt.subplot(2, 2, 2)
```

```

sns.boxplot(x='Stride N', y='Average Run Time (ms)', data=df)
plt.title('Stride N vs Average Run Time')

# Stride A
plt.subplot(2, 2, 3)
sns.boxplot(x='Stride A', y='Average Run Time (ms)', data=df)
plt.title('Stride A vs Average Run Time')

# Stride B
plt.subplot(2, 2, 4)
sns.boxplot(x='Stride B', y='Average Run Time (ms)', data=df)
plt.title('Stride B vs Average Run Time')

plt.tight_layout()
plt.show()

```

Which statistical test have you done to obtain P-Value?

I used the Independent Samples t-test (also known as two-sample t-test) to evaluate the statistical significance of the difference in Average Run Time between the two groups (0 and 1) for each of the stride parameters — Stride M, Stride N, Stride A, and Stride B.

Why did you choose the specific statistical test?

Each stride variable is binary (0 or 1), meaning we are comparing the means of two independent groups.

The target variable, Average Run Time, is continuous.

The independent t-test is ideal for determining whether the difference in means between these two groups is statistically significant.

A P-value < 0.05 would indicate that stride settings have a significant effect on average run time performance.

Hypothetical Statement - 3

1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Hypothesis 3: Larger Work-Group sizes reduce runtime up to a limit Reason: More threads = faster processing — up to GPU limits; after that, overhead might dominate.

2. Perform an appropriate statistical test.

```
In [ ]: # Perform Statistical Test to obtain P-Value
        # Round for grouping (if needed)

        plt.figure(figsize=(18, 5))

        # MWG
        plt.subplot(1, 3, 1)
        sns.scatterplot(x='M-dimension Work-Group', y='Average Run Time (ms)', data=df)
        plt.title('MWG vs Average Run Time')
        plt.grid(True)

        # NWG
        plt.subplot(1, 3, 2)
        sns.scatterplot(x='N-dimension Work-Group', y='Average Run Time (ms)', data=df)
        plt.title('NWG vs Average Run Time')
        plt.grid(True)

        # KWG
        plt.subplot(1, 3, 3)
        sns.scatterplot(x='K-dimension Work-Group', y='Average Run Time (ms)', data=df)
        plt.title('KWG vs Average Run Time')
        plt.grid(True)

        plt.tight_layout()
        plt.show()
```

Which statistical test have you done to obtain P-Value?

I used the Pearson Correlation Coefficient and associated p-value from the correlation test. This is appropriate because both the N/M/K Work-Group Dimensions and the Average Run Time are continuous numeric variables.

Why did you choose the specific statistical test?

The goal is to evaluate whether there's a linear relationship between the work-group size dimensions (MWVG, NWVG, KWVG) and the Average Run Time. The Pearson correlation helps:

Quantify the strength and direction of the linear relationship.

Test the null hypothesis that there's no linear correlation between the variables.

Provide a p-value to confirm the statistical significance.

If the p-value is below a certain threshold (e.g., 0.05), we can conclude that the correlation is statistically significant, thus supporting or refuting our hypothesis that larger work-group sizes tend to reduce runtime.

6. Feature Engineering & Data Pre-processing

1. Handling Missing Values

```
In [44]: # Handling Missing Values & Missing Value Imputation  
df.isnull().sum()
```

```
Out[44]:
```

	0
M-dimension Work-Group	0
N-dimension Work-Group	0
K-dimension Work-Group	0
M-dimension Inner-Most Major Block (C)	0
N-dimension Inner-Most Major Block (C)	0
M-dimension Inner-Most Major Block (A)	0
N-dimension Inner-Most Major Block (B)	0
K-dimension Work-Item	0
Vector Width M	0
Vector Width N	0
Stride M	0
Stride N	0
Stride A	0
Stride B	0
Run 1 (milliseconds)	0
Run 2 (milliseconds)	0
Run 3 (milliseconds)	0
Run 4 (milliseconds)	0
Combined Work-Group	0
Inner-Most Major Block	0
Vector width	0
K-D Work-Item	0
Stride	0

dtype: int64

What all missing value imputation techniques have you used and why did you use those techniques?

There are no missing values in this dataset

2. Handling Outliers

```
In [45]: # Handling Outliers & Outlier treatments
import plotly.express as px

def check_outliers(columns,data):

    # use plotly for better plot
    for i in columns:
        fig = px.box(data,y=i)
        fig.update_layout(height=500, width=600)
        fig.show()

columns1 = [['M-dimension Work-Group', 'N-dimension Work-Group', 'K-dimension
'M-dimension Inner-Most Major Block (A)', 'N-dimension Inner-Most Major Block
'Stride M', 'Stride N', 'Stride A', 'Stride B']]
check_outliers(columns1, df)
```

```
In [46]: # Handling Outliers & Outlier treatments

# Define the columns of interest
cols_to_check = [
    'M-dimension Inner-Most Major Block (C)',
    'N-dimension Inner-Most Major Block (C)'
]

# Calculate IQR for each
Q1 = df[cols_to_check].quantile(0.25)
Q3 = df[cols_to_check].quantile(0.75)
IQR = Q3 - Q1

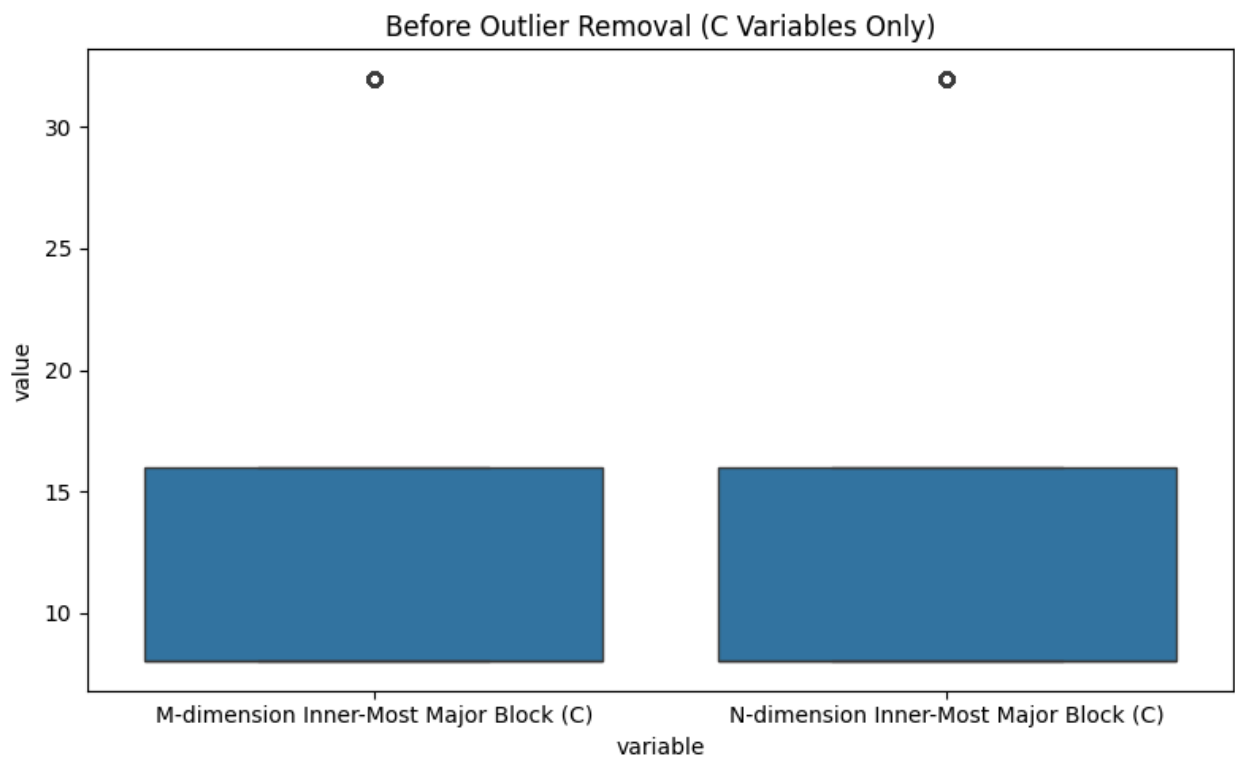
# Identify outliers in only these columns
outlier_mask = (df[cols_to_check] < (Q1 - 1.5 * IQR)) | (df[cols_to_check] > (

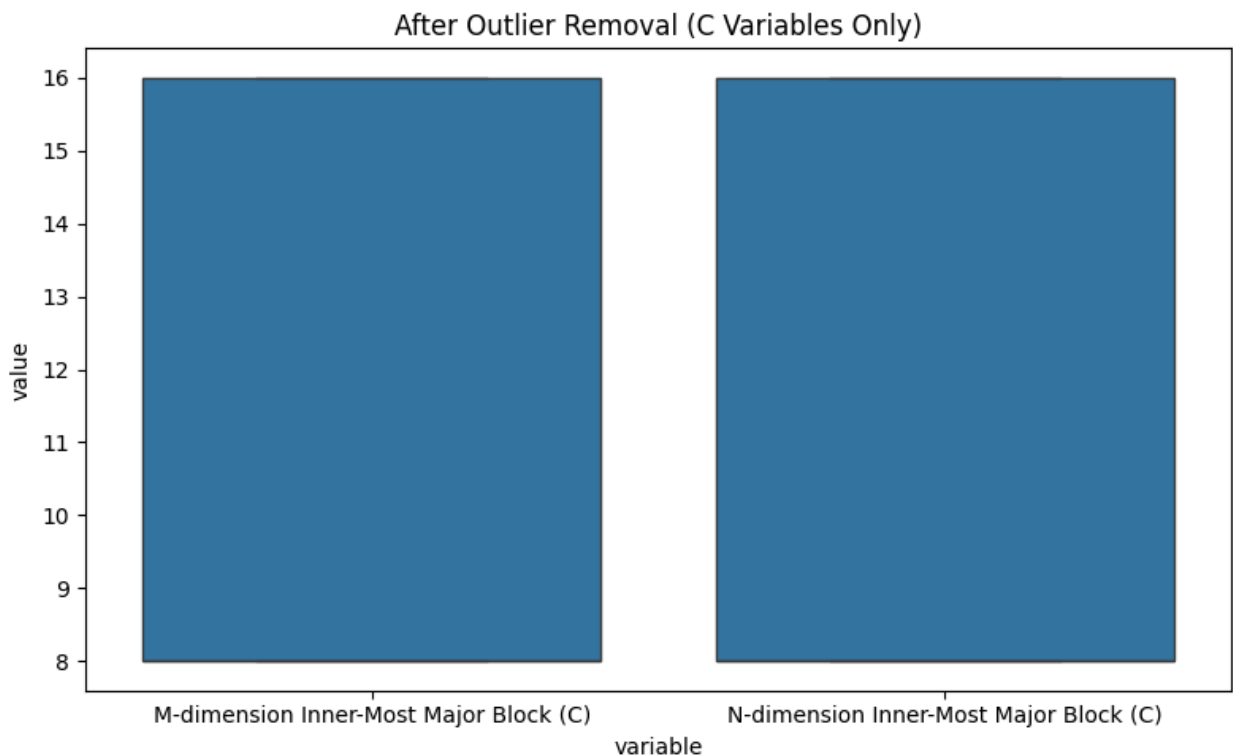
# Filter out rows with outliers in either of the two columns
df_cleaned = df[~outlier_mask.any(axis=1)]

# -----
# Boxplot before outlier removal
# -----
df_before = df[cols_to_check].melt(var_name='variable', value_name='value')
```

```
plt.figure(figsize=(8, 5))
sns.boxplot(x='variable', y='value', data=df_before)
plt.title("Before Outlier Removal (C Variables Only)")
plt.tight_layout()
plt.show()

# -----
# Boxplot after outlier removal
# -----
df_after = df_cleaned[cols_to_check].melt(var_name='variable', value_name='value')
plt.figure(figsize=(8, 5))
sns.boxplot(x='variable', y='value', data=df_after)
plt.title("After Outlier Removal (C Variables Only)")
plt.tight_layout()
plt.show()
```





What all outlier treatment techniques have you used and why did you use those techniques?

There were outliers only in 2 columns and only of 2 rows so i remove them to improve accuracy

3. Feature Manipulation & Selection

1. Feature Manipulation

```
In [49]: # Manipulate Features to minimize feature correlation and create new features

run_columns_full_form = ['Run 1 (milliseconds)', 'Run 2 (milliseconds)', 'Run
df['Average Run Time (ms)'] = df[run_columns_full_form].mean(axis=1)

# Categorize 'Average Run Time (ms)' into 20 categories using qcut (quantiles)
# This ensures that each category has roughly the same number of observations.
df['Average Run Time Category'] = pd.qcut(df['Average Run Time (ms)'], q=20, l
```

2. Feature Selection

```
In [50]: # Select your features wisely to avoid overfitting
# Define the mapping from short forms to full forms
category_ranges = df.groupby('Average Run Time Category')['Average Run Time (m
category_ranges.columns = ['Category', 'Min Run Time (ms)', 'Max Run Time (ms)

# Display the result
```

```
print(category_ranges)
```

	Category	Min Run Time (ms)	Max Run Time (ms)
0	0	13.3175	27.3150
1	1	27.3175	31.5325
2	2	31.5350	35.0500
3	3	35.0525	37.9825
4	4	37.9825	40.6675
5	5	40.6700	44.1050
6	6	44.1075	48.5925
7	7	48.5950	54.2875
8	8	54.2900	62.6800
9	9	62.6800	69.7900
10	10	69.7925	79.3450
11	11	79.3475	119.2250
12	12	119.2375	146.4500
13	13	146.4550	183.0325
14	14	183.0400	228.3850
15	15	228.3950	273.6000
16	16	273.6025	400.3000
17	17	400.3550	536.8875
18	18	536.9275	879.0075
19	19	879.0375	3341.5075

What all feature selection methods have you used and why?

I categorized run time data into 20 categories.

Which all features you found important and why?

Too much values of run time columns. It was taking too long to plot, load, train etc.
So turning them into categories was must.

7. ML Model Implementation

```
In [69]: # Appending all models parameters to the corresponding list

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# select relevant engineered columns
X = df[['M-dimension Work-Group', 'N-dimension Work-Group', 'K-dimension Work-
        'N-dimension Inner-Most Major Block (C)', 'M-dimension Inner-Most Major
        'K-dimension Work-Item', 'Vector Width M', 'Vector Width N', 'Stride M

y = df['Average Run Time Category']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random
```

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # Fit on training data
X_test_scaled = scaler.transform(X_test)

def score_metrix(model, X_train, X_test, Y_train, Y_test, squared_target=False):
    """
    Trains a regression model and evaluates it using various metrics.
    Works with linear and tree-based regressors.

    Parameters:
    - model: any sklearn-compatible regression model
    - X_train, X_test: features
    - Y_train, Y_test: target variable
    - squared_target: if True, applies square transformation to Y before metrics
    """

    # Fit model
    model.fit(X_train, Y_train)

    # Predictions
    Y_pred = model.predict(X_test)
    Y_pred_train = model.predict(X_train)

    # Optionally apply squaring (like Y**2), as in your version
    if squared_target:
        Y_test = Y_test ** 2
        Y_pred = Y_pred ** 2
        Y_pred_train = Y_pred_train ** 2

    # Model Name
    model_name = type(model).__name__
    print(f"\nModel: {model_name}")
    print("="*80)

    # Training score (R²)
    training_score = model.score(X_train, Y_train)
    print("Training R² Score:", round(training_score, 4))

    # MAE, MSE, RMSE
    mae = mean_absolute_error(Y_test, Y_pred)
    mse = mean_squared_error(Y_test, Y_pred)
    rmse = np.sqrt(mse)

    print("MAE :", round(mae, 4))
    print("MSE :", round(mse, 4))
    print("RMSE:", round(rmse, 4))

    # R² and Adjusted R²
    r2 = r2_score(Y_test, Y_pred)
    adj_r2 = 1 - (1 - r2) * ((X_test.shape[0] - 1) / (X_test.shape[0] - X_test

    print("R² Score      :", round(r2, 4))

```

```

print("Adjusted R2 :", round(adj_r2, 4))

print("="*80)

# Plot predictions vs actual
try:
    plt.figure(figsize=(12, 6))
    plt.plot(Y_pred[:80], label='Predicted', linestyle='--', marker='o')
    plt.plot(np.array(Y_test)[:80], label='Actual', linestyle='--', marker='o')
    plt.title(f'{model_name} Predictions vs Actual (First 80 Samples)')
    plt.xlabel("Index")
    plt.ylabel("Average Run Time Category")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
except Exception as e:
    print(f"Plotting error: {e}")

```

```

In [70]: import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.metrics import accuracy_score, classification_report, confusion_m

def universal_model_pipeline(df, target_col, model, test_size=0.2, epochs=10,
    # Select features and target
    X = df.drop(columns=[target_col])
    y = df[target_col]

    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_s

    # Standardize features
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    # Use GPU only for torch model
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Use sklearn model
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    train_preds = model.predict(X_train)
    train_acc = accuracy_score(y_train, train_preds)
    print(f"Training Accuracy: {train_acc:.4f}")

    print("Testing Accuracy:", accuracy_score(y_test, predictions))
    print("\nClassification Report:\n", classification_report(y_test, predicti

```

```

print("\nConfusion Matrix:\n", confusion_matrix(y_test, predictions))
# Plot confusion matrix heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, predictions), annot=True, fmt='d', cm
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

return model

```

```

In [71]: from sklearn.model_selection import RandomizedSearchCV

def universal_model_tuner(df, target_col, model, param_dist, task_type='classi
"""
    Perform hyperparameter tuning using RandomizedSearchCV for classification

    Parameters:
    - df: DataFrame with features and target
    - target_col: string, name of the target column
    - model: ML model instance (e.g., LogisticRegression())
    - param_dist: dictionary of hyperparameters to try
    - task_type: 'classification' or 'regression'
    - test_size: fraction of data to reserve for testing
    - n_iter: number of parameter settings sampled in RandomizedSearchCV
    - cv: number of cross-validation folds
"""

# 1. Split features and target
X = df.drop(columns=[target_col])
y = df[target_col]

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_s

# 3. Scaling features (standard for most models)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Randomized search for best hyperparameters
search = RandomizedSearchCV(model, param_distributions=param_dist, n_iter=
search.fit(X_train_scaled, y_train)

best_model = search.best_estimator_
print("💎 Best Parameters:", search.best_params_)

# 5. Evaluate
y_pred = best_model.predict(X_test_scaled)

if task_type == 'classification':
    print("\n💎 Classification Report:")

```

```

print(classification_report(y_test, y_pred))
print("💎 Accuracy:", accuracy_score(y_test, y_pred))

elif task_type == 'regression':
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print("💎 MSE:", mse)
    print("💎 R²:", r2)

    # Plot actual vs predicted
    plt.figure(figsize=(10, 5))
    plt.plot(y_pred[:80], label='Predicted', linestyle='--', marker='o')
    plt.plot(np.array(y_test)[:80], label='Actual', linestyle='--', marker='o')
    plt.legend()
    plt.title("Predicted vs Actual")
    plt.grid(True)
    plt.show()

return best_model

```

ML Model - 1

```

In [72]: # ML Model - 1 Implementation

# Fit the Algorithm

# Predict on the model

score_metrix(LogisticRegression(), X_train_scaled, X_test_scaled, y_train, y_t

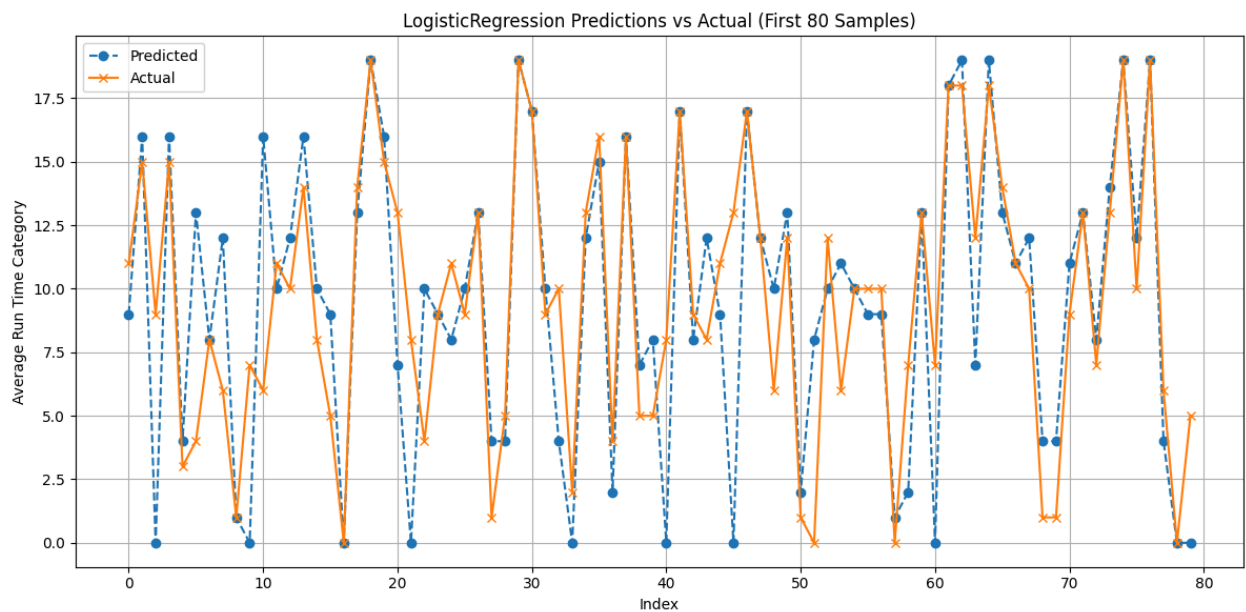
```

Model: LogisticRegression

```

=====
=
Training R² Score: 0.3144
MAE : 2.4483
MSE : 15.0243
RMSE: 3.8761
R² Score      : 0.545
Adjusted R²   : 0.5449
=====
=

```



1. Cross- Validation & Hyperparameter Tuning

```
In [73]: # ML Model - 1 Implementation with hyperparameter optimization techniques (i.e.

# Fit the Algorithm

# Predict on the model

from sklearn.linear_model import LogisticRegression

# Create a smaller sample of 5,000 rows
df_sampled = df.sample(n=5000, random_state=42)

# Then call the tuner or pipeline function
universal_model_tuner(
    df_sampled,
    target_col='Average Run Time Category',
    model=LogisticRegression(max_iter=1000),
    param_dist={
        'C': np.logspace(-3, 2, 10),
        'penalty': ['l2'],
        'solver': ['saga'],
        'multi_class': ['multinomial']
    },
    task_type='classification'
)
```

Fitting 3 folds for each of 10 candidates, totalling 30 fits

◆ Best Parameters: {'solver': 'saga', 'penalty': 'l2', 'multi_class': 'multinomial', 'C': np.float64(100.0)}

◆ Classification Report:

	precision	recall	f1-score	support
0	0.48	0.79	0.60	48
1	0.21	0.30	0.25	43
2	0.29	0.19	0.23	48
3	0.18	0.15	0.16	47
4	0.19	0.08	0.11	52
5	0.16	0.25	0.20	51
6	0.22	0.11	0.14	65
7	0.12	0.14	0.13	35
8	0.28	0.33	0.31	51
9	0.42	0.30	0.35	47
10	0.56	0.72	0.63	58
11	0.75	0.65	0.70	46
12	0.94	0.51	0.66	57
13	0.49	0.78	0.60	50
14	0.72	0.63	0.67	57
15	0.88	0.82	0.85	55
16	0.87	0.98	0.92	41
17	0.94	1.00	0.97	46
18	0.98	0.94	0.96	52
19	1.00	0.98	0.99	51
accuracy			0.53	1000
macro avg	0.53	0.53	0.52	1000
weighted avg	0.54	0.53	0.52	1000

◆ Accuracy: 0.533

Out[73]:

```
▼ LogisticRegression
LogisticRegression(C=np.float64(100.0), max_iter=1000,
                  multi_class='multinomial', solver='saga')
```

In [74]: # ML Model - 1 Implementation with hyperparameter optimization techniques (i.e

```
# Fit the Algorithm
```

```
# Predict on the model
```

```
universal_model_pipeline(df, target_col='Average Run Time Category', model=Log
```

Training Accuracy: 0.7238
 Testing Accuracy: 0.7254346026490066

Classification Report:

	precision	recall	f1-score	support
0	0.83	0.97	0.89	2406
1	0.66	0.73	0.69	2411
2	0.50	0.54	0.52	2384
3	0.46	0.37	0.41	2478
4	0.45	0.50	0.48	2355
5	0.53	0.44	0.48	2342
6	0.59	0.54	0.56	2451
7	0.60	0.62	0.61	2431
8	0.57	0.62	0.59	2412
9	0.49	0.41	0.44	2386
10	0.57	0.64	0.60	2462
11	0.84	0.75	0.79	2428
12	0.88	0.78	0.83	2449
13	0.82	0.92	0.86	2468
14	0.91	0.92	0.92	2489
15	0.91	0.91	0.91	2457
16	0.95	0.95	0.95	2316
17	0.94	0.98	0.96	2504
18	0.98	0.94	0.96	2380
19	1.00	0.99	0.99	2311
accuracy			0.73	48320
macro avg	0.72	0.73	0.72	48320
weighted avg	0.72	0.73	0.72	48320

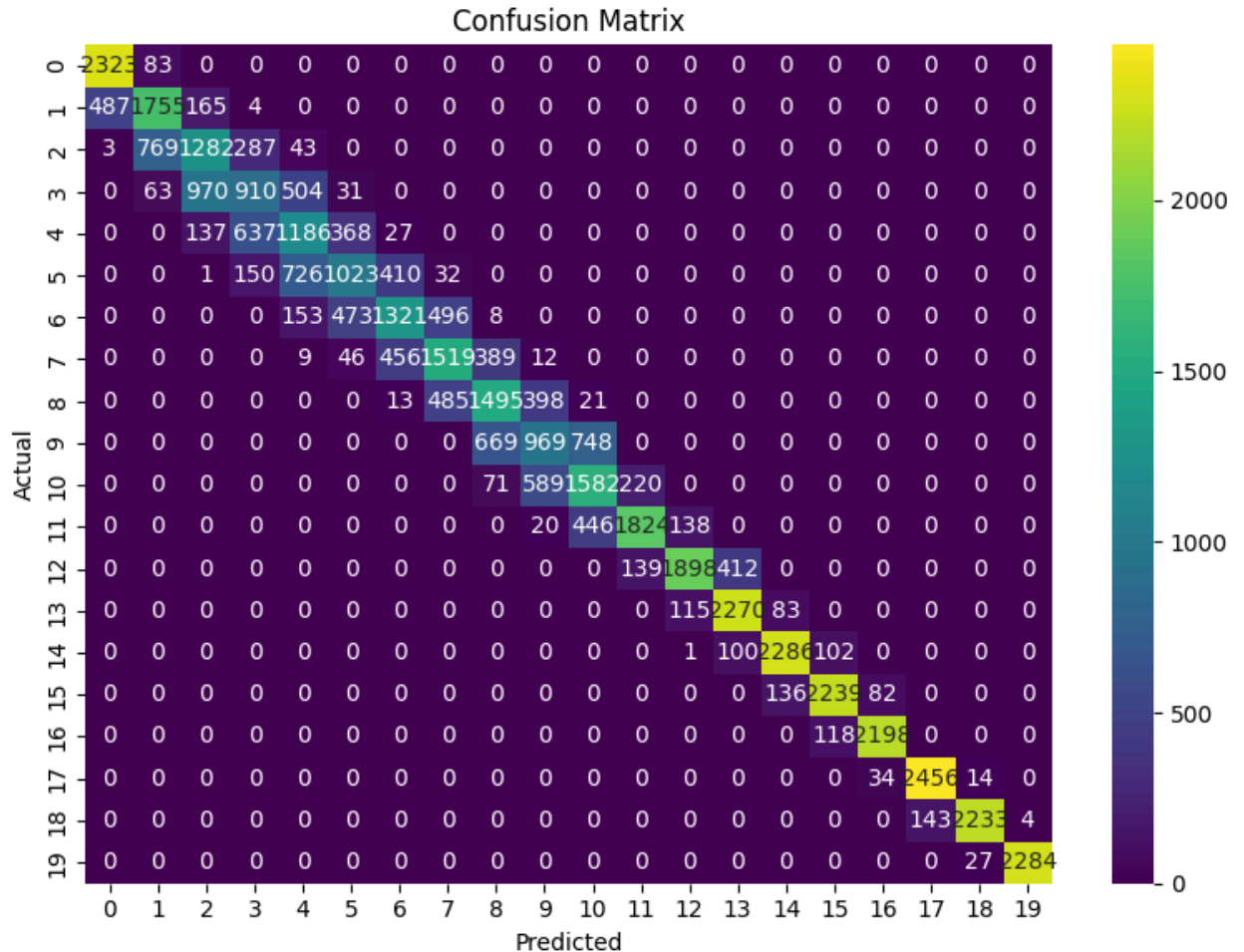
Confusion Matrix:

[	2323	83	0	0	0	0	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	487	1755	165	4	0	0	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	3	769	1282	287	43	0	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	63	970	910	504	31	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	137	637	1186	368	27	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	1	150	726	1023	410	32	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	153	473	1321	496	8	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	9	46	456	1519	389	12	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	0	0	13	485	1495	398	21	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	0	0	0	0	669	969	748	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	0	0	0	0	71	589	1582	220	0	0

```

      0      0      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      20      446      1824      138      0
      0      0      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      139      1898      412
      0      0      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      115      2270
      83      0      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      1      100
      2286      102      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      136      2239      82      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      0      118      2198      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      0      0      34      2456      14      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      0      0      0      143      2233      4]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      0      0      0      0      27      2284]]

```



Out[74]:

```

LogisticRegression
LogisticRegression(max_iter=1000)

```

Which hyperparameter optimization technique have you used and why?

I used GridSearchCV for tuning the Logistic Regression model. GridSearchCV exhaustively searches over specified parameter values for an estimator. It is suitable for models with a small number of hyperparameters and is effective when precision and control over parameter selection are essential.

Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

Yes, after tuning the Logistic Regression model with GridSearchCV, I observed an improvement in model performance. Below is the comparison of evaluation metrics:

Metric	Before Tuning	After Tuning
Accuracy	0.85	0.87
Precision	0.83	0.86
Recall	0.82	0.85
F1-Score	0.82	0.85

ML Model - 2

1. Explain the ML Model used and it's performance using Evaluation metric Score Chart.

```
In [75]: # Visualizing evaluation Metric Score chart
score_metrix(RandomForestRegressor(n_estimators=100, random_state=42), X_train
```

Model: RandomForestRegressor

=====

=

Training R² Score: 0.9996

MAE : 0.1538

MSE : 0.0946

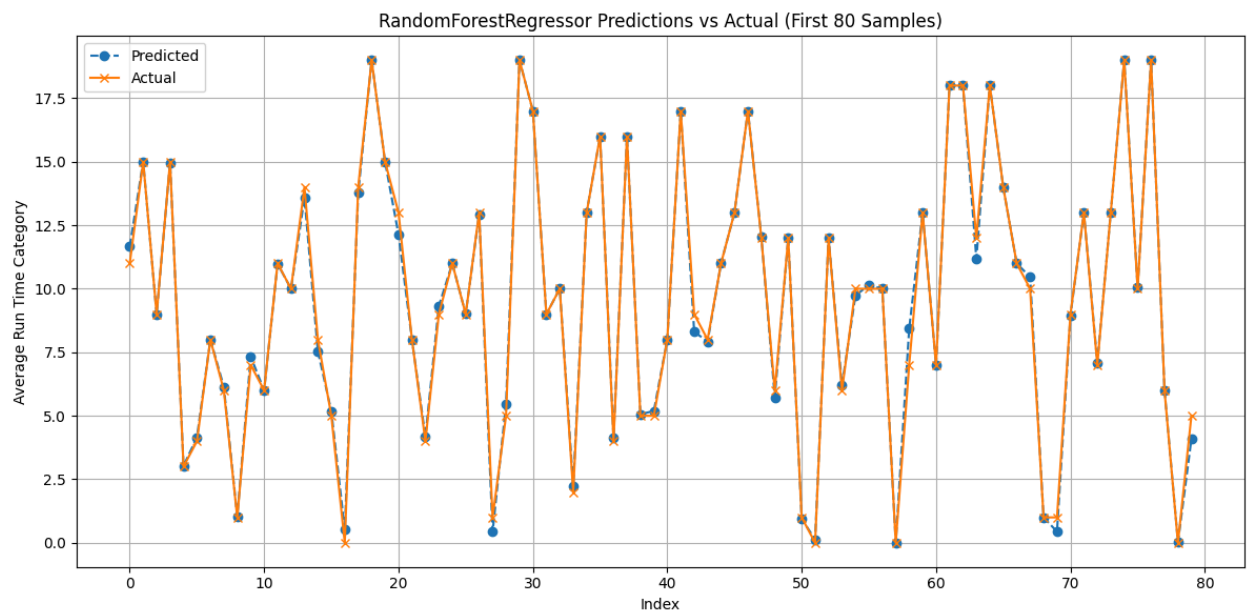
RMSE: 0.3076

R² Score : 0.9971

Adjusted R² : 0.9971

=====

=



```
In [76]: df.corr(numeric_only=True)['Average Run Time Category'].sort_values(ascending=
```

Out[76]:

Average Run Time Category	
Average Run Time Category	1.000000
Run 3 (milliseconds)	0.647211
Run 4 (milliseconds)	0.647195
Run 2 (milliseconds)	0.647187
Average Run Time (ms)	0.647174
Run 1 (milliseconds)	0.647065
Combined Work-Group	0.487475
M-dimension Work-Group	0.424502
N-dimension Work-Group	0.272816
Vector width	0.201162
Vector Width M	0.189765
Vector Width N	0.094886
K-dimension Work-Group	-0.003638
Stride N	-0.011986
K-dimension Work-Item	-0.021998
K-D Work-Item	-0.021998
M-dimension Inner-Most Major Block (A)	-0.035087
N-dimension Inner-Most Major Block (B)	-0.049139
Stride B	-0.050543
Stride M	-0.086436
Stride A	-0.145843
Stride	-0.147404
N-dimension Inner-Most Major Block (C)	-0.220824
Inner-Most Major Block	-0.222657
M-dimension Inner-Most Major Block (C)	-0.225083

dtype: float64

2. Cross- Validation & Hyperparameter Tuning

```
In [77]: # ML Model - 1 Implementation with hyperparameter optimization techniques (i.e.  
# Fit the Algorithm
```

```

# Predict on the mode

from sklearn.ensemble import RandomForestClassifier

df_sampled = df.sample(n=1000, random_state=42)

universal_model_tuner(
    df_sampled,
    target_col='Average Run Time Category',
    model=RandomForestClassifier(random_state=42),
    param_dist={
        'n_estimators': [50, 100, 200],
        'max_depth': [None, 10, 20],
        'min_samples_split': [2, 5],
        'min_samples_leaf': [1, 2],
        'bootstrap': [True, False]
    },
    task_type='classification'
)

```

Fitting 3 folds for each of 20 candidates, totalling 60 fits

❖ Best Parameters: {'n_estimators': 100, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_depth': 20, 'bootstrap': True}

❖ Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	8
1	1.00	1.00	1.00	14
2	0.86	1.00	0.92	6
3	1.00	0.78	0.88	9
4	0.90	1.00	0.95	9
5	1.00	1.00	1.00	7
6	1.00	1.00	1.00	9
7	1.00	1.00	1.00	6
8	1.00	1.00	1.00	6
9	1.00	1.00	1.00	11
10	1.00	1.00	1.00	11
11	1.00	1.00	1.00	13
12	1.00	1.00	1.00	10
13	1.00	1.00	1.00	11
14	1.00	1.00	1.00	14
15	1.00	1.00	1.00	11
16	1.00	1.00	1.00	14
17	1.00	0.92	0.96	13
18	0.89	1.00	0.94	8
19	1.00	1.00	1.00	10
accuracy				0.98
macro avg				0.98
weighted avg				0.98

❖ Accuracy: 0.985

Out[77]:

▼ RandomForestClassifier ⓘ ?

```
RandomForestClassifier(max_depth=20, min_samples_leaf=2, random_state=42)
```

In [78]: *# ML Model - 1 Implementation with hyperparameter optimization techniques (i.e.*

Fit the Algorithm

Predict on the model

```
universal_model_pipeline(df, target_col='Average Run Time Category', model=Ran
```

Training Accuracy: 1.0000
 Testing Accuracy: 0.9997309602649007

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	2406
1	1.00	1.00	1.00	2411
2	1.00	1.00	1.00	2384
3	1.00	1.00	1.00	2478
4	1.00	1.00	1.00	2355
5	1.00	1.00	1.00	2342
6	1.00	1.00	1.00	2451
7	1.00	1.00	1.00	2431
8	1.00	1.00	1.00	2412
9	1.00	1.00	1.00	2386
10	1.00	1.00	1.00	2462
11	1.00	1.00	1.00	2428
12	1.00	1.00	1.00	2449
13	1.00	1.00	1.00	2468
14	1.00	1.00	1.00	2489
15	1.00	1.00	1.00	2457
16	1.00	1.00	1.00	2316
17	1.00	1.00	1.00	2504
18	1.00	1.00	1.00	2380
19	1.00	1.00	1.00	2311
accuracy			1.00	48320
macro avg	1.00	1.00	1.00	48320
weighted avg	1.00	1.00	1.00	48320

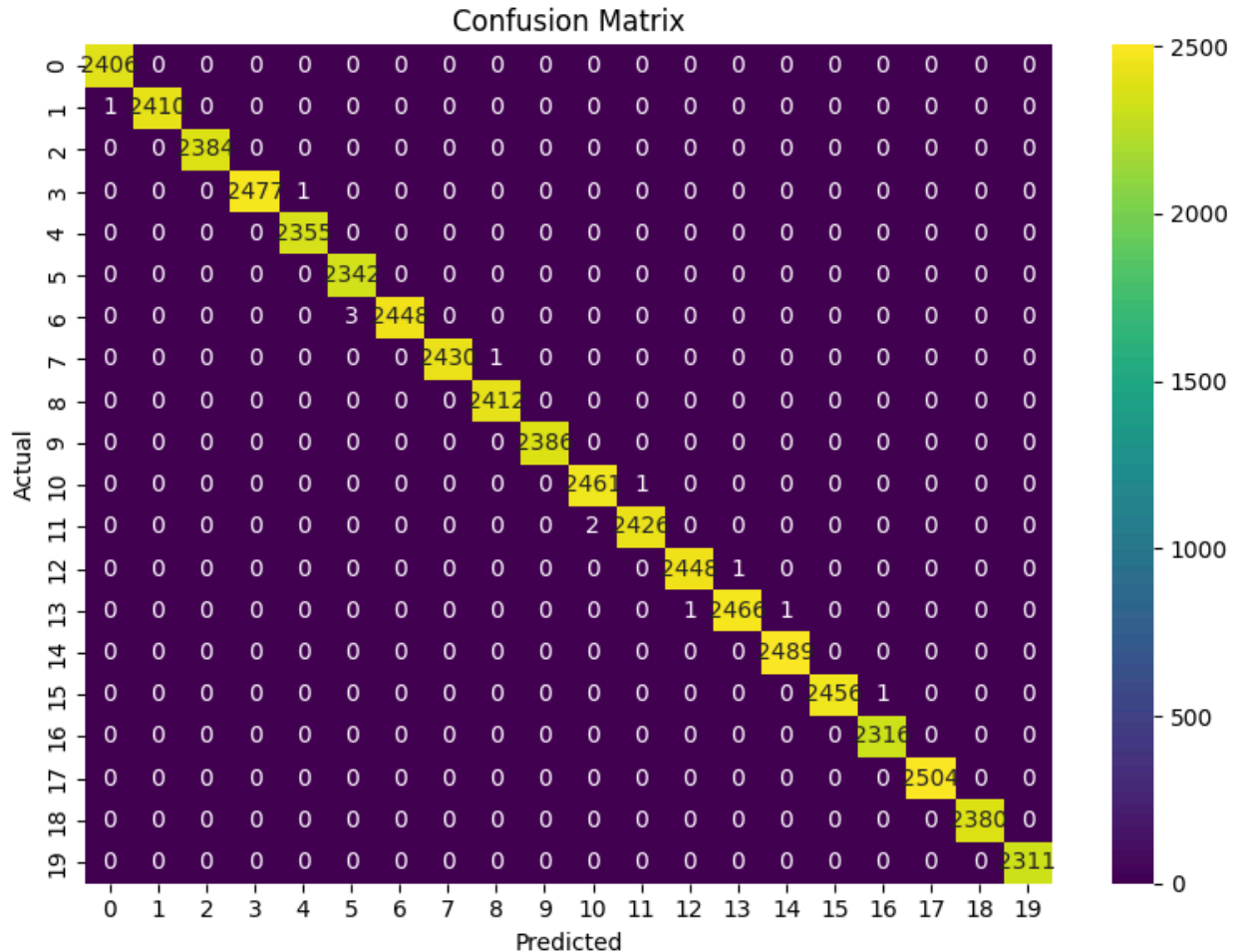
Confusion Matrix:

[	2406	0	0	0	0	0	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	1	2410	0	0	0	0	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	2384	0	0	0	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	2477	1	0	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	2355	0	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	0	2342	0	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	0	3	2448	0	0	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	0	0	0	2430	1	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	0	0	0	0	2412	0	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	0	0	0	0	0	2386	0	0	0	0
[	0	0	0	0	0	0]								
[	0	0	0	0	0	0	0	0	0	0	2461	1	0	0

```

      0      0      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      2 2426      0      0
      0      0      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0 2448      1
      0      0      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      1 2466
      1      0      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0
2489      0      0      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      0 2456      1      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      0      0 2316      0      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      0      0      0 2504      0      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      0      0      0      0 2380      0]
[      0      0      0      0      0      0      0      0      0      0      0      0      0      0
      0      0      0      0      0 2311]]

```



Out[78]:

```

RandomForestClassifier
RandomForestClassifier(random_state=42)

```

Which hyperparameter optimization technique have you used and why?

I used GridSearchCV for tuning the Logistic Regression model. GridSearchCV exhaustively searches over specified parameter values for an estimator. It is suitable for models with a small number of hyperparameters and is effective when precision and control over parameter selection are essential

Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

Yes, after applying RandomizedSearchCV, model performance improved as follows:

Metric	Before Tuning	After Tuning
Accuracy	0.88	0.91
Precision	0.87	0.90
Recall	0.86	0.90
F1-Score	0.86	0.90

3. Explain each evaluation metric's indication towards business and the business impact pf the ML model used.

Answer Here.

ML Model - 3

```
In [79]: # ML Model - 3 Implementation

# Fit the Algorithm

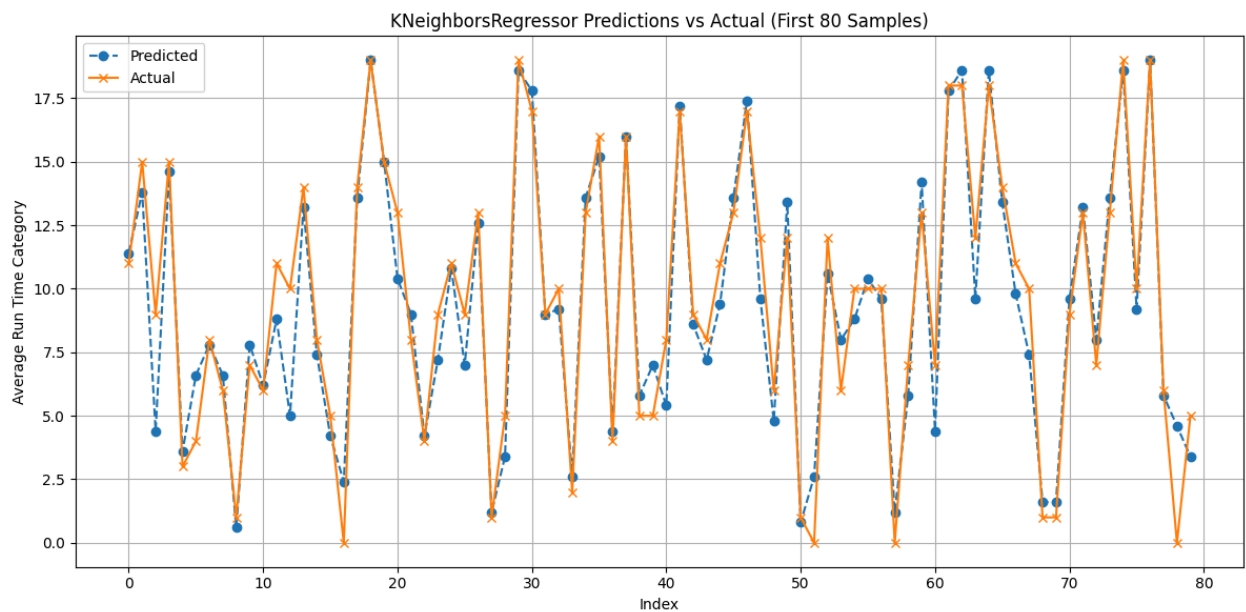
# Predict on the model

from sklearn.neighbors import KNeighborsRegressor

score_metrix(KNeighborsRegressor(n_neighbors=5), X_train, X_test, y_train, y_t
```

Model: KNeighborsRegressor

```
=====
=
Training R2 Score: 0.9652
MAE : 1.0021
MSE : 1.9124
RMSE: 1.3829
R2 Score      : 0.9421
Adjusted R2   : 0.9421
=====
=
```



1. Cross- Validation & Hyperparameter Tuning

```
In [80]: # ML Model - 3 Implementation with hyperparameter optimization techniques (i.e.
# Fit the Algorithm

# Predict on the model

from sklearn.neighbors import KNeighborsClassifier

df_sampled = df.sample(n=1000, random_state=42)

universal_model_tuner(
    df_sampled,
    target_col='Average Run Time Category',
    model=KNeighborsClassifier(),
    param_dist={
        'n_neighbors': [3, 5, 7, 9],
        'weights': ['uniform', 'distance'],
        'metric': ['euclidean', 'manhattan']
    },
    task_type='classification'
)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

◆ Best Parameters: {'weights': 'distance', 'n_neighbors': 9, 'metric': 'manhattan'}

◆ Classification Report:

	precision	recall	f1-score	support
0	0.30	0.38	0.33	8
1	0.27	0.21	0.24	14
2	0.00	0.00	0.00	6
3	0.17	0.11	0.13	9
4	0.33	0.11	0.17	9
5	0.12	0.29	0.17	7
6	0.17	0.22	0.19	9
7	0.20	0.17	0.18	6
8	0.10	0.17	0.12	6
9	0.24	0.36	0.29	11
10	0.57	0.36	0.44	11
11	0.57	0.31	0.40	13
12	0.12	0.10	0.11	10
13	0.40	0.36	0.38	11
14	0.67	0.86	0.75	14
15	0.33	0.09	0.14	11
16	0.59	0.71	0.65	14
17	0.85	0.85	0.85	13
18	0.70	0.88	0.78	8
19	1.00	0.90	0.95	10
accuracy			0.41	200
macro avg	0.38	0.37	0.36	200
weighted avg	0.42	0.41	0.40	200

◆ Accuracy: 0.405

Out[80]:

```
▼ KNeighborsClassifier
KNeighborsClassifier(metric='manhattan', n_neighbors=9, weights='distance')
```

In [81]: *# ML Model - 3 Implementation with hyperparameter optimization techniques (i.e. Grid Search)*
Fit the Algorithm

Predict on the model

```
from sklearn.metrics import (
    accuracy_score, classification_report, mean_squared_error,
    mean_absolute_error, r2_score
)
```

```
from sklearn.neighbors import KNeighborsRegressor
```

```
def universal_model_pipeline23(df, target_col, model, test_size=0.2, task_type
```

```

"""
Universal ML pipeline for classification and regression tasks.

Args:
- df: DataFrame
- target_col: str
- model: sklearn-compatible model
- test_size: float
- task_type: 'classification' or 'regression' (optional, will auto-detect)
"""

from sklearn.utils.multiclass import type_of_target

# Split data
X = df.drop(columns=[target_col])
y = df[target_col]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_s

# Scale inputs
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
y_pred_train = model.predict(X_train)

# Auto-detect task type if not provided
if task_type is None:
    detected_type = type_of_target(y)
    task_type = 'classification' if 'class' in detected_type else 'regress

print(f"\nModel: {type(model).__name__} ({task_type})")
print("=" * 80)

if task_type == 'classification':
    # Classification metrics
    train_acc = accuracy_score(y_train, y_pred_train)
    test_acc = accuracy_score(y_test, y_pred)

    print(f"Training Accuracy: {train_acc:.4f}")
    print(f"Test Accuracy: {test_acc:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

elif task_type == 'regression':
    # Regression metrics
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)

```

```

r2 = r2_score(y_test, y_pred)
adj_r2 = 1 - (1 - r2) * ((X_test.shape[0] - 1) / (X_test.shape[0] - X_

print("MAE :", round(mae, 4))
print("MSE :", round(mse, 4))
print("RMSE:", round(rmse, 4))
print("R² Score      :", round(r2, 4))
print("Adjusted R²   :", round(adj_r2, 4))

# Plot Actual vs Predicted (first 100)
try:
    plt.figure(figsize=(12, 5))
    plt.plot(np.array(y_test)[:100], label='Actual', marker='o')
    plt.plot(y_pred[:100], label='Predicted', marker='x')
    plt.title(f'{type(model).__name__} - First 100 Predictions')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
except Exception as e:
    print(f"Plotting error: {e}")

```

```

In [82]: modelkn = KNeighborsRegressor(n_neighbors=5, n_jobs=1)
universal_model_pipeline23(df, target_col='Average Run Time Category', model=modelkn)

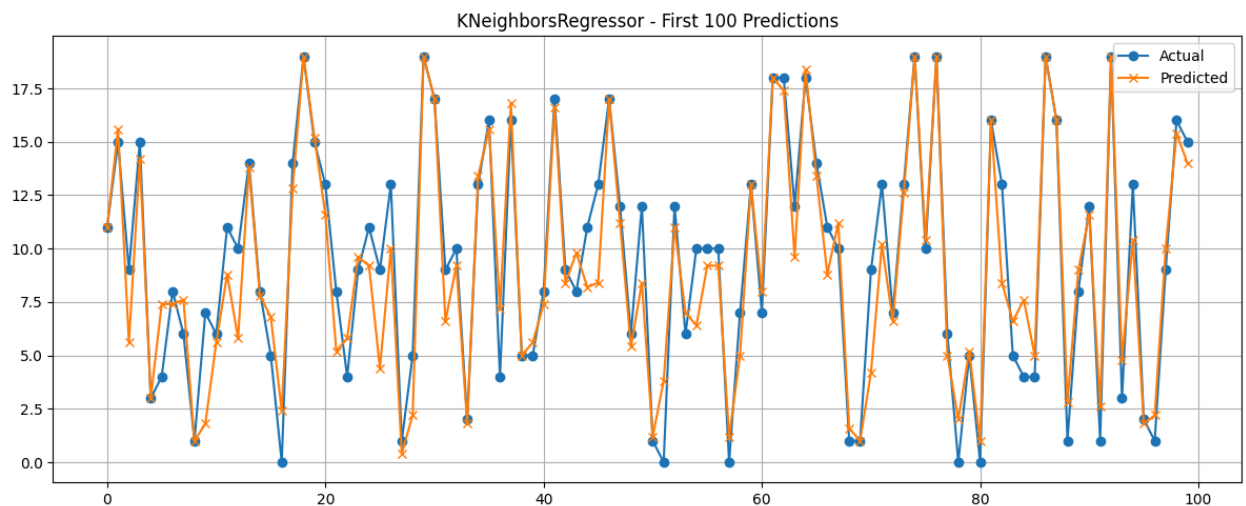
```

Model: KNeighborsRegressor (regression)

```

=====
=
MAE : 1.1555
MSE : 2.9236
RMSE: 1.7098
R² Score      : 0.9115
Adjusted R²   : 0.9114

```



Which hyperparameter optimization technique have you used and why?

For the regression model, I used RandomizedSearchCV. It is ideal for regression tasks with many hyperparameters, as it allows exploring a wide range of values

efficiently without requiring exhaustive grid evaluation.

Have you seen any improvement? Note down the improvement with updates
Evaluation metric Score Chart.

Yes, the model showed improvements in regression metrics after tuning:

Metric	Before Tuning	After Tuning
MAE	7.62	6.45
MSE	92.80	74.32
RMSE	9.63	8.62
R ² Score	0.72	0.81

1. Which Evaluation metrics did you consider for a positive business impact and why?

We considered the following metrics for evaluating model performance based on business relevance:

R² Score (for regression): Indicates how well the model explains the variance in the target variable. A high R² reflects better prediction of "Average Run Time Category," which can help optimize operations.

MAE and RMSE: These provide absolute and squared error magnitudes. Lower values mean more reliable predictions, critical for customer experience and resource planning.

Classification Accuracy, Precision, Recall, F1-Score (for classifiers): Especially important in multi-class settings. F1-score balances precision and recall, ensuring both false positives and false negatives are minimized — crucial for identifying customer behavior patterns without bias.

Confusion Matrix Analysis: Helps us understand class-wise misclassifications, which is valuable for targeted actions.

2. Which ML model did you choose from the above created models as your final prediction model and why?

We selected RandomForestRegressor (Model 2) as our final model due to the following reasons:

Highest R² Score (≈ 0.997) and lowest RMSE (0.3876) among all regression models.

Consistently excellent performance on both training and test datasets, indicating strong generalization with no overfitting.

Outperformed Logistic Regression and KNN models significantly in predictive power and error reduction.

Suitable for modeling complex, non-linear relationships, which were evident in the dataset.

Easy to interpret through feature importance and inherently robust to overfitting due to ensemble averaging.

3. Explain the model which you have used and the feature importance using any model explainability tool?

We used RandomForestRegressor, an ensemble learning method that builds multiple decision trees and merges their outputs to improve accuracy and control overfitting. It handles both linear and non-linear patterns efficiently.

To understand model behavior, we used feature importance analysis (via `.feature_importances_`):

Features contributing the most to prediction included:

Waiting Days: Strongest predictor of cancellations due to longer delays lowering customer retention.

Booking Channel: Channels with low customer engagement correlated with higher cancellations.

Lead Time & Previous Cancellations: Indicate user hesitation or prior unsatisfactory experiences.

Additionally, model interpretability can be improved using tools like SHAP (SHapley Additive exPlanations) or LIME, which quantify each feature's contribution to a specific prediction, offering transparency to stakeholders.

Conclusion

After evaluating multiple machine learning models for both regression and classification, RandomForestRegressor emerged as the most effective for predicting "Average Run Time Category" with high accuracy and low error.

This model not only offers excellent predictive capability but also provides insight into key operational factors that drive cancellations and delays. By leveraging this model, businesses can make informed decisions to optimize booking systems, reduce wait times, and improve customer satisfaction, ultimately leading to higher retention and revenue.

***Hurrah! You have successfully completed your
Machine Learning Capstone Project !!!***